

JAVA 17 OCP · YDS STUDY SERIES

Unit 05 · Methods · Complete Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu düzenlenebilir ana kaynak, verilen PDF bölümünün sayfa sırasını eksiksiz izler. Her kaynak satırı, PDF çıkarımının paragraf sınırlarını güvenilir biçimde taşımadığı yerlerde içerik kaybını önlemek için ayrı English → Türkçe çifti olarak tutulmuştur. Java kodu çevrilmeden ve yinelenmeden gösterilir.

[Vocabulary](#) · [Grammar notes](#)

Kaynak ve kapsam özeti

Alan	Değer
Bölüm	Chapter 5 — Methods
Kaynak PDF sayfaları	0219–0274
Beklenen kaynak sayfa sayısı	56
Korunan içerik	Başlıklar, paragraflar/satırlar, kod, tablolar, şekiller, callout'lar, Summary, Exam Essentials ve Review Questions
Çıkarılan içerik	Yalnız tekrarlanan running header/footer ve sayfa numarası
Kaynak dışı içerik	Belgenin sonundaki açıkça etiketlenmiş teknik pekiştirme appendix'i

İçindekiler

- [Designing Methods](#)
- [Declaring Local and Instance Variables](#)
- [Working with Varargs](#)
- [Applying Access Modifiers](#)
- [Accessing `static` Data](#)
- [static Imports](#)
- [Passing Data among Methods](#)
- [Overloading Methods](#)
- [Summary](#)
- [Exam Essentials](#)
- [Review Questions](#)

Kaynak metin ve çeviri

Source page 0219

English: Chapter

Türkçe: Bölüm

English: 5
Türkçe: 5

English: Methods
Türkçe: Method'lar

OCF EXAM OBJECTIVES COVERED IN

Türkçe başlık: OCF SINAVININ HEDEFLERİ

English: THIS CHAPTER:
Türkçe: BU BÖLÜM:

English: [x] Utilizing Java Object-Oriented Approach
Türkçe: [x] Java Nesne Yönelimli Yaklaşımının Kullanımı

English: • Create classes and records, and define and use instance and static fields and methods, constructors, and instance and static initializers
Türkçe: • Class ve record oluşturun; instance ve `static` field'ları, method'ları, constructor'ları, instance initializer'ları ve `static` initializer'ları tanımlayıp kullanın

English: • Implement overloading, including vararg methods
Türkçe: • Varargs method'lar dahil olmak üzere overloading uygulayın

Source page 0220

English: In previous chapters, you learned how to write snippets of code without much thought about the methods that contained the code. In this chapter, you explore methods in depth including modifiers, arguments, varargs, overloading, and autoboxing. Many of these fundamentals, such as access and static modifiers, are applicable to classes and other types throughout the rest of the book. If you're having difficulty, you might want to read this chapter twice!

Türkçe: Önceki bölümlerde, kodu barındıran method'lar üzerinde fazla düşünmeden kod parçaları yazmayı öğrendiniz. Bu bölümde modifier, argument, varargs, overloading ve autoboxing dahil olmak üzere method'ları ayrıntılı biçimde inceleyeceksiniz. Access modifier ve `static` modifier gibi bu temel kuralların çoğu, kitabın geri kalanındaki class'lar ve diğer type'lar için de geçerlidir. Zorlanırsanız bu bölümü iki kez okumak isteyebilirsiniz!

Designing Methods

Türkçe başlık: Method Tasarlama

English: Every interesting Java program we've seen has had a `main()` method. You can write other methods too. For example, you can write a basic method to take a nap, as shown in Figure 5.1.

Türkçe: Şimdiye kadar gördüğümüz her ilgi çekici Java programında bir `main()` method'u vardı. Başka method'lar da yazabilirsiniz. Örneğin, Şekil 5.1'de gösterildiği gibi uyumak için basit bir method yazabilirsiniz.

English: FIGURE 5.1 Method declaration

Türkçe: ŞEKİL 5.1 Method bildirimi

English: Access modifier

Türkçe: Access modifier

English: Method name

Türkçe: Method adı

English: Parentheses (required)

Türkçe: Parantez (gerekli)

English: Optional specifier

Türkçe: Optional specifier (isteğe bağlı belirteç)

English: Return type

Türkçe: Return type

English: Exception (optional)

Türkçe: Exception (isteğe bağlı)

English: List of parameters

Türkçe: Parameter listesi

English: Method body

Türkçe: Method gövdesi

```
public final void nap(int minutes) throws InterruptedException {  
    // take a nap  
}
```

English: This is called a method declaration, which specifies all the information needed to call the method. There are a lot of parts, and we cover each one in more detail. Two of the parts—the method name and parameter list—are called the method signature. The method signature provides instructions for how callers can reference this method. The method signature does not include the return type and access modifiers, which control where the method can be referenced.

Türkçe: Buna method declaration (method bildirimi) denir; method'u çağırmak için gereken bütün bilgileri belirtir. Birçok parçadan oluşur ve her birini ayrıntılı inceleyeceğiz. Bu parçalardan ikisi—method name ve parameter list—method signature'ı (method imzasını) oluşturur. Method signature, çağıran kodun bu method'a nasıl başvuracağını gösterir. Method'un nereden çağrılabilceğini etkileyen return type ve access modifier, method signature'a dahil değildir.

English: Table 5.1 is a brief reference to the elements of a method declaration. Don't worry if it seems like a lot of information— by the time you finish this chapter, it will all fit together.

Türkçe: Tablo 5.1, bir method declaration'ın öğelerini kısaca gösterir. Bilgi fazla görünüyorsa endişelenmeyin; bölümü bitirdiğinizde parçalar birbiriyle bütünleşecektir.

Source page 0221

English: TABLE 5.1 Parts of a method declaration in Figure 5.1

Türkçe: TABLO 5.1 Şekil 5.1'deki method bildiriminin bölümleri

English: Element · Value in `nap()` example · Required?

Türkçe: Öğe · `nap()` örneğindeki değer · Gerekli mi?

English: Access modifier · `public` · No

Türkçe: Access modifier · `public` · Hayır

English: Optional specifier · `final` · No

Türkçe: Optional specifier · `final` · Hayır

English: Return type · `void` · Yes

Türkçe: Return type · `void` · Evet

English: Method name · `nap` · Yes

Türkçe: Method name · `nap` · Evet

English: Parameter list · `(int minutes)` · Yes, but can be empty parentheses

Türkçe: Parameter listesi · `(int minutes)` · Evet, ancak parantezlerin içi boş olabilir

English: Method signature · `nap(int minutes)` · Yes

Türkçe: Method signature · `nap(int minutes)` · Evet

English: Exception list · `throws InterruptedException` · No
Türkçe: Exception list · `throws InterruptedException` · Hayır

English: Method body · `{ // take a nap }` · Yes, except for abstract methods
Türkçe: Method body · `{ // take a nap }` · Evet; `abstract` method'lar hariç

English: To call this method, just use the method signature and provide an int value in parentheses:
Türkçe: Bu method'u çağırmak için method signature'ı kullanın ve parantez içinde bir `int` value geçirin:

```
nap(10);
```

English: Let's start by taking a look at each of these parts of a basic method.
Türkçe: Temel method'un bu bölümlerinin her birine göz atarak başlayalım.

Access Modifiers

Türkçe başlık: Access Modifier'lar

English: An access modifier determines what classes a method can be accessed from. Think of it like a security guard. Some classes are good friends, some are distant relatives, and some are complete strangers. Access modifiers help to enforce when these components are allowed to talk to each other. Java offers four choices of access modifier:

Türkçe: Bir access modifier, bir method'a hangi class'lardan erişilebileceğini belirler. Bunu bir güvenlik görevlisi gibi düşünün: Bazı class'lar yakın arkadaş, bazıları uzak akraba, bazılarıysa tamamen yabancıdır. Access modifier'lar, bu bileşenlerin ne zaman iletişim kurabileceğini belirleyen kuralları uygular. Java dört access modifier seçeneği sunar:

English: `private` The `private` modifier means the method can be called only from within the same class.

Türkçe: `private`: Method yalnızca aynı class içinden çağrılabilir.

English: Package Access With package access, the method can be called only from a class in the same package. This one is tricky because there is no keyword. You simply omit the access modifier. Package access is sometimes referred to as package-private or default access (even within this book!).

Türkçe: Package access: Method yalnızca aynı package içindeki bir class'tan çağrılabilir. Burada bir keyword bulunmaması işi biraz zorlaştırır; access modifier'ı yazmazsınız. Package access, package-private veya default access olarak da adlandırılır (bu kitapta bile!).

Source page 0222

English: `protected` The `protected` modifier means the method can be called only from a class in the same package or a subclass.

Türkçe: `protected`: Method yalnızca aynı package içindeki bir class'tan veya bir subclass'tan çağrılabilir.

English: public The public modifier means the method can be called from anywhere.

Türkçe: `public`: Method her yerden çağrılabilir.

English: For simplicity, we're primarily concerned with access modifiers applied to methods and fields in this chapter. Rules for access modifiers are also applicable to classes and other types you learn about in Chapter 7, "Beyond Classes," such as interfaces, enums, and records.

Türkçe: Basitlik için bu bölümde öncelikle method ve field'lara uygulanan access modifier'larla ilgileniyoruz. Bu kurallar class'lar ile Chapter 7, "Beyond Classes" bölümünde göreceğiniz interface, enum ve record gibi diğer type'lar için de geçerlidir.

English: We explore the impact of the various access modifiers later in this chapter. For now, just master identifying valid syntax of methods. The exam creators like to trick you by putting method elements in the wrong order or using incorrect values.

Türkçe: Bu bölümün ilerleyen kısımlarında farklı access modifier'ların etkisini inceleyeceğiz. Şimdilik method bildirimlerinin geçerli syntax'ını tanımayı iyice öğrenin. Sınavda method öğeleri yanlış sıraya konarak veya geçersiz değerler kullanılarak sizi yanıltmaya çalışabilirler.

English: We'll see practice examples as we go through each of the method elements in this chapter.

Türkçe: Bu bölümdeki method öğelerinin her birini incelerken uygulama örneklerini göreceğiz.

English: Make sure you understand why each of these is a valid or invalid method declaration. Pay attention to the access modifiers as you figure out what is wrong with the ones that don't compile when inserted into a class:

Türkçe: Her birinin neden geçerli veya geçersiz bir method declaration olduğunu anladığınızdan emin olun. Bunlar bir class içine eklendiğinde derlenmeyen bildirimlerde neyin yanlış olduğunu bulurken access modifier'lara özellikle dikkat edin:

```
public class ParkTrip {
    public void skip1() {}
    default void skip2() {} // DOES NOT COMPILE
    void public skip3() {} // DOES NOT COMPILE
    void skip4() {}
}
```

English: The skip1() method is a valid declaration with public access. The skip4() method is a valid declaration with package access. The skip2() method doesn't compile because default is not a valid access modifier. There is a default keyword, which is used in switch statements and interfaces, but default is never used as an access modifier.

Türkçe: `skip1()` `public` access'e, `skip4()` ise package access'e sahip geçerli bildirimlerdir. `default` geçerli bir access modifier olmadığı için `skip2()` derlenmez. `default` keyword'ü switch statement'larda ve interface'lerde kullanılır; ancak hiçbir zaman access modifier olarak kullanılmaz.

English: The skip3() method doesn't compile because the access modifier is specified after the return type.

Türkçe: Access modifier return type'tan sonra yazıldığı için `skip3()` derlenmez.

Optional Specifiers

Türkçe başlık: İsteğe Bağlı Belirleyiciler

English: There are a number of optional specifiers for methods, shown in Table 5.2. Unlike with access modifiers, you can have multiple specifiers in the same method (although not all combinations are legal). When this happens, you can specify them in any order. And since these specifiers are optional, you are allowed to not have any of them at all. This means you can have zero or more specifiers in a method declaration.

Türkçe: Method'lar için Tablo 5.2'de gösterilen birçok optional specifier vardır. Access modifier'lardan farklı olarak aynı method'da birden fazla specifier bulunabilir; ancak her kombinasyon geçerli değildir. Birden fazla specifier kullanıldığında bunlar herhangi bir sırada yazılabilir. Hepsi isteğe bağlı olduğu için bir method declaration sıfır veya daha fazla specifier içerebilir.

English: As you can see in Table 5.2, four of the method modifiers are covered in later chapters, and the last two aren't even in scope for the exam (and are seldom used in real life). In this chapter, we focus on introducing you to these modifiers. Using them often requires a lot more rules.

Türkçe: Tablo 5.2'de görebileceğiniz gibi, method modifier'lardan dördü sonraki bölümlerde ele alınmaktadır ve son ikisi sınavın kapsamında bile değildir (ve gerçek hayatta nadiren kullanılır). Bu bölümde sizi bu modifier'larla tanıştırmaya odaklanıyoruz. Bunları kullanmak genellikle çok daha fazla kural gerektirir.

Source page 0223

English: TABLE 5.2 Optional specifiers for methods

Türkçe: TABLO 5.2 Method'lar için optional specifier'lar

English: Modifier · Description · Chapter covered

Türkçe: Modifier · Açıklama · Ele alındığı chapter

English: `static` · Indicates the method is a member of the shared class object · Chapter 5

Türkçe: `static` · Method'un paylaşılan class object'inin bir member'ı olduğunu belirtir · Chapter 5

English: `abstract` · Used in an abstract class or interface when the method body is excluded · Chapter 6

Türkçe: `abstract` · Method body bulunmadığında abstract class veya interface içinde kullanılır · Chapter 6

English: `final` · Specifies that the method may not be overridden in a subclass · Chapter 6

Türkçe: `final` · Method'un bir subclass'ta override edilemeyeceğini belirtir · Chapter 6

English: `default` · Used in an interface to provide a default implementation of a method for classes that implement the interface · Chapter 7

Türkçe: `default` · Interface'i implement eden class'lara bir method'un default implementation'ını sağlamak için interface içinde kullanılır · Chapter 7

English: `synchronized` · Used with multithreaded code · Chapter 13

Türkçe: `synchronized` · Multithreaded code ile kullanılır · Chapter 13

English: `native` · Used when interacting with code written in another language, such as C++ · Out of scope

Türkçe: `native` · C++ gibi başka bir dilde yazılmış code ile etkileşim kurulurken kullanılır · Kapsam dışı

English: `strictfp` · Used for making floating-point calculations portable · Out of scope

Türkçe: `strictfp` · Floating-point hesaplamaları taşınabilir hâle getirmek için kullanılır · Kapsam dışı

English: While access modifiers and optional specifiers can appear in any order, they must all appear before the return type. In practice, it is common to list the access modifier first. As you'll also learn in upcoming chapters, some specifiers are not compatible with one another. For example, you can't declare a method (or class) both `final` and `abstract`.

Türkçe: Access modifier'lar ve optional specifier'lar kendi aralarında herhangi bir sırada bulunabilse de hepsi return type'tan önce gelmelidir. Uygulamada access modifier'ı önce yazmak yaygındır. Ayrıca ileride göreceğiniz gibi bazı specifier'lar birbiriyle uyumlu değildir. Örneğin bir method (veya class) hem `final` hem `abstract` bildirilemez.

English: Remember, access modifiers and optional specifiers can be listed in any order, but once the return type is specified, the rest of the parts of the method are written in a specific order: name, parameter list, exception list, body.

Türkçe: Access modifier ve optional specifier'ların herhangi bir sırada yazılabileceğini unutmayın. Ancak return type belirtildikten sonra method'un geri kalanı belirli bir sırayı izler: method name, parameter list, exception list ve method body.

English: Again, just focus on syntax for now. Do you see why these compile or don't compile?

Türkçe: Yine, şimdilik sadece sözdizimine odaklanın. Bunların neden derlendiğini veya derlenmediğini görüyor musunuz?

```
public class Exercise {
    public void bike1() {}
    public final void bike2() {}
    public static final void bike3() {}
    public final static void bike4() {}
    public modifier void bike5() {}           // DOES NOT COMPILE
    public void final bike6() {}             // DOES NOT COMPILE
    final public void bike7() {}
}
```

Source page 0224

English: The `bike1()` method is a valid declaration with no optional specifier. This is okay— it is optional, after all. The `bike2()` method is a valid declaration, with `final` as the optional specifier. The `bike3()` and `bike4()` methods are valid declarations with both `final` and `static` as optional specifiers. The order of these two keywords doesn't matter.

Türkçe: `bike1()`, optional specifier içermeyen geçerli bir bildirimdir; zaten specifier isteğe bağlıdır. `bike2()`, optional specifier olarak `final` kullanan geçerli bir bildirimdir. `bike3()` ve `bike4()` ise hem `final` hem `static` kullanan geçerli bildirimlerdir. Bu iki keyword'ün sırası önemli değildir.

English: The `bike5()` method doesn't compile because modifier is not a valid optional specifier.

Türkçe: `modifier` geçerli bir optional specifier olmadığından `bike5()` method'u derlenmez.

English: The `bike6()` method doesn't compile because the optional specifier is after the return type.

Türkçe: Optional specifier return type'tan sonra yazıldığı için `bike6()` method'u derlenmez.

English: The `bike7()` method does compile. Java allows the optional specifiers to appear before the access modifier. This is a weird case and not one you need to know for the exam. We are mentioning it so you don't get confused when practicing.

Türkçe: `bike7()` method'u derlenir. Java, optional specifier'ların access modifier'dan önce gelmesine izin verir. Bu sıra alışılmadık ve sınav için bilmeniz gerekmez; alıştırma yaparken şaşırmamanız için belirtilmiştir.

Return Type

Türkçe başlık: Return Type

English: The next item in a method declaration is the return type. It must appear after any access modifiers or optional specifiers and before the method name. The return type might be an actual Java type such as `String` or `int`. If there is no return type, the `void` keyword is used.

Türkçe: Method declaration'daki sonraki öğe return type'tır. Access modifier ve optional specifier'lardan sonra, method name'den önce gelmelidir. Return type `String` veya `int` gibi gerçek bir Java type'i olabilir. Method bir değer döndürmüyorsa `void` keyword'ü kullanılır.

English: This special return type comes from the English language: `void` means without contents.

Türkçe: Bu özel return type'ın adı İngilizceden gelir: `void`, “içeriksiz/boş” anlamındadır.

English: Remember that a method must have a return type. If no value is returned, the `void` keyword must be used. You cannot omit the return type.

Türkçe: Bir method'un mutlaka return type'a sahip olması gerektiğini unutmayın. Herhangi bir value döndürülüyorsa `void` keyword'ü kullanılmalıdır; return type tamamen atlanamaz.

English: When checking return types, you also have to look inside the method body. Methods with a return type other than void are required to have a return statement inside the method body. This return statement must include the primitive or object to be returned. Methods that have a return type of void are permitted to have a return statement with no value returned or omit the return statement entirely. Think of a return statement in a void method as the method saying, “I’m done!” and quitting early, such as the following:

Türkçe: Return type'ları denetlerken method body'ye de bakmalısınız. Return type'ı `void` dışında olan method'larda, body içinde döndürülecek primitive veya object'i içeren bir `return` statement bulunmalıdır. Return type'ı `void` olan method'larda değersiz bir `return` statement bulunabilir ya da `return` tamamen atlanabilir. `void` method'daki `return` statement'ı, method'un “İşim bitti!” deyip aşağıdaki gibi erken çıkması olarak düşünün:

```
public void swim(int distance) {
    if(distance <= 0) {
        // Exit early, nothing to do!
        return;
    }
    System.out.print("Fish is swimming " + distance + " meters");
}
```

English: Ready for some examples? Can you explain why these methods compile or don't?

Türkçe: Birkaç örneğe hazır mısınız? Bu method'ların neden derlenip derlenmediğini açıklayabilir misiniz?

```
public class Hike {
    public void hike1() {}
    public void hike2() { return; }
    public String hike3() { return ""; }
    public String hike4() {} // DOES NOT COMPILE
    public hike5() {} // DOES NOT COMPILE
    public String hike6() { } // DOES NOT COMPILE
}
```

Source page 0225

```
String hike7(int a) { // DOES NOT COMPILE
    if (1 < 2) return "orange";
}
}
```

English: Since the return type of the `hike1()` method is void, the return statement is optional.

Türkçe: `hike1()` method'unun return type'ı `void` olduğundan `return` statement isteğe bağlıdır.

English: The hike2() method shows the optional return statement that correctly doesn't return anything. The hike3() method is a valid declaration with a String return type and a return statement that returns a String. The hike4() method doesn't compile because the return statement is missing. The hike5() method doesn't compile because the return type is missing. The hike6() method doesn't compile because it attempts to use two return types.

Türkçe: hike2(), doğru biçimde hiçbir değer döndürmeyen isteğe bağlı return statement'ı gösterir. hike3(), String return type'a ve String döndüren bir return statement'a sahip geçerli bir bildirimdir. hike4() bir return statement içermediği için, hike5() return type içermediği için, hike6() ise iki return type kullanmaya çalıştığı için derlenmez.

English: You get only one return type.

Türkçe: Yalnızca tek bir return type kullanabilirsiniz.

English: The hike7() method is a little tricky. There is a return statement, but it doesn't always get run. Even though 1 is always less than 2, the compiler won't fully evaluate the if statement and requires a return statement if this condition is false. What about this modified version?

Türkçe: hike7() biraz tuzaklıdır. Bir return statement vardır ama her execution path'te çalışmaz. 1 her zaman 2'den küçük olsa da compiler bu if statement'ını return-path analizi için tamamen değerlendirmez; condition false olursa da bir value döndürülmesini ister. Peki şu değiştirilmiş sürüm?

```
String hike8(int a) {  
    if (1 < 2) return "orange";  
    return "apple"; // COMPILER WARNING  
}
```

English: The code compiles, although the compiler will produce a warning about unreachable code (or dead code). This means the compiler was smart enough to realize you wrote code that cannot possibly be reached.

Türkçe: Compiler unreachable code (erişilemeyen kod veya dead code) hakkında uyarı üretse de kod derlenir. Bu, compiler'ın hiçbir zaman ulaşamayacak kod yazdığınızı anlayabildiği anlamına gelir.

Java 17 doğrulama notu: Kaynağın İngilizce iddiası yukarıda eksiksiz korunmuştur; ancak Temurin/OpenJDK 17'de javac -Xlint:all bu hike8() örneği için warning üretmez. Java'nın reachability kuralları, if (1 < 2) sonrasındaki return "apple"; statement'ını compile-time açısından reachable kabul eder. Kod başarıyla derlenir.

English: When returning a value, it needs to be assignable to the return type. Can you spot what's wrong with two of these examples?

Türkçe: Bir değer döndürülürken, return type'a atanabilir olması gerekir. Bu örneklerden ikisinde neyin yanlış olduğunu görebiliyor musunuz?

```
public class Measurement {  
    int getHeight1() {  
        int temp = 9;  
        return temp;  
    }  
    int getHeight2() {  
        int temp = 9L; // DOES NOT COMPILE  
        return temp;  
    }  
    int getHeight3() {  
        long temp = 9L;  
        return temp; // DOES NOT COMPILE  
    }  
}
```

English: The getHeight2() method doesn't compile because you can't assign a long to an int.

Türkçe: getHeight2() method'u derlenmez; çünkü long value int variable'a atanamaz.

English: The method getHeight3() method doesn't compile because you can't return a long value as an int. If this wasn't clear to you, you should go back to Chapter 2, "Operators," and reread the sections about numeric types and casting.

Türkçe: getHeight3() derlenmez; çünkü long bir value, int olarak döndürülemez. Bu açık değilse Chapter 2, "Operators" bölümündeki numeric type ve casting kısımlarını yeniden okuyun.

Editor note: Basılı kaynakta "The method getHeight3() method" biçimindeki tekrarlı ifade aynen korunmuştur; doğal anlamı hemen altındaki Türkçe çeviride verilmiştir.

Source page 0226

Method Name

Türkçe başlık: Method Adı

English: Method names follow the same rules we practiced with variable names in Chapter 1, "Building Blocks." To review, an identifier may only contain letters, numbers, currency symbols, or _. Also, the first character is not allowed to be a number, and reserved words are not allowed. Finally, the single underscore character is not allowed.

Türkçe: Method name'ler, Chapter 1, "Building Blocks" bölümünde variable name'ler için uyguladığımız kurallara uyar. Hatırlatmak gerekirse bir identifier yalnızca harf, rakam, currency symbol veya _ içerebilir. İlk karakter rakam olamaz; reserved word'ler ve tek başına _ identifier olarak kullanılamaz.

English: By convention, methods begin with a lowercase letter, but they are not required to. Since this is a review of Chapter 1, we can jump right into practicing with some examples:

Türkçe: Convention gereği method name küçük harfle başlar; ancak Java bunu zorunlu tutmaz. Chapter 1'i gözden geçirdiğimize göre birkaç örnekle doğrudan alıştırmaya geçebiliriz:

```
public class BeachTrip {
    public void jog1() {}
    public void 2jog() {}           // DOES NOT COMPILE
    public jog3 void() {}          // DOES NOT COMPILE
    public void Jog_$( ) {}
    public _() {}                  // DOES NOT COMPILE
    public void() {}              // DOES NOT COMPILE
}
```

English: The jog1() method is a valid declaration with a traditional name. The 2jog() method doesn't compile because identifiers are not allowed to begin with numbers. The jog3() method doesn't compile because the method name is before the return type. The Jog_\$() method is a valid declaration. While it certainly isn't good practice to start a method name with a capital letter and end with punctuation, it is legal. The _ method is not allowed since it consists of a single underscore. The final line of code doesn't compile because the method name is missing.

Türkçe: jog1(), convention'a uygun bir name taşıyan geçerli bir method declaration'dır. Identifier rakamla başlayamayacağı için 2jog() derlenmez. Method name return type'tan önce yazıldığı için jog3() de derlenmez. Jog_\$() geçerli bir declaration'dır; method name'in büyük harfle başlayıp punctuation ile bitmesi iyi bir uygulama olmasa da Java açısından geçerlidir. Yalnızca tek underscore'dan oluşan _, method name olarak kullanılamaz. Son satırda method name eksik olduğu için o da derlenmez.

Parameter List

Türkçe başlık: Parameter Listesi

English: Although the parameter list is required, it doesn't have to contain any parameters. This means you can just have an empty pair of parentheses after the method name, as follows:

Türkçe: Parameter listesi gerekli olmasına rağmen herhangi bir parameter içermesine gerek yoktur. Bu, method adından sonra aşağıdaki gibi yalnızca bir çift boş parantez kullanabileceğiniz anlamına gelir:

```
public class Sleep {
    void nap() {}
}
```

English: If you do have multiple parameters, you separate them with a comma. There are a couple more rules for the parameter list that you'll see when we cover varargs shortly. For now, let's practice looking at method declaration with "regular" parameters:

Türkçe: Birden fazla parameter varsa bunları virgülle ayırırsınız. Az sonra varargs'ı ele alırken parameter list için birkaç kural daha göreceksiniz. Şimdilik "normal" parameter'lara sahip method declaration'lar üzerinde alıştırmaya bakalım:

```
public class PhysicalEducation {
    public void run1() {}
    public void run2 {} // DOES NOT COMPILE
    public void run3(int a) {}
    public void run4(int a; int b) {} // DOES NOT COMPILE
    public void run5(int a, int b) {}
}
```

Source page 0227

English: The `run1()` method is a valid declaration without any parameters. The `run2()` method doesn't compile because it is missing the parentheses around the parameter list. The `run3()` method is a valid declaration with one parameter. The `run4()` method doesn't compile because the parameters are separated by a semicolon rather than a comma. Semicolons are for separating statements, not for parameter lists. The `run5()` method is a valid declaration with two parameters.

Türkçe: `run1()` parameter içermeyen geçerli bir bildirimdir. `run2()`, parameter list'i çevreleyen parantezler eksik olduğu için derlenmez. `run3()` tek parameter'lı geçerli bir bildirimdir. `run4()` ise parameter'lar virgül yerine noktalı virgülle ayrıldığı için derlenmez; noktalı virgül statement'ları ayırır, parameter list'lerini değil. `run5()` iki parameter'lı geçerli bir bildirimdir.

Method Signature

Türkçe başlık: Method Signature

English: A method signature, composed of the method name and parameter list, is what Java uses to uniquely determine exactly which method you are attempting to call. Once it determines which method you are trying to call, it then determines if the call is allowed. For example, attempting to access a private method outside the class or assigning the return value of a void method to an int variable results in compiler errors. Neither of these compiler errors is related to the method signature, though.

Türkçe: Method name ile parameter list'ten oluşan method signature, Java'nın tam olarak hangi method'u çağırmak istediğinizi benzersiz biçimde belirlemesini sağlar. Java önce hedef method'u belirler, ardından çağrıya izin verilip verilmediğini denetler. Örneğin class dışından `private` method'a erişmek veya `void` method'un dönüşünü `int` variable'a atamak compiler error üretir; ancak bu hataların hiçbirisi method signature ile ilgili değildir.

English: It's important to note that the names of the parameters in the method signature are not used as part of a method signature. The parameter list is about the types of parameters and their order. For example, the following two methods have the exact same signature:

Türkçe: Method signature belirlenirken parameter name'lerin dikkate alınmadığına dikkat edin. Parameter list'i parameter type'ları ve bunların sırası belirler. Örneğin aşağıdaki iki method tamamen aynı signature'a sahiptir:

```
public class Trip {
    public void visitZoo(String name, int waitTime) {}
    public void visitZoo(String attraction, int rainFall) {} // DOES NOT COMPILE
}
```

English: Despite having different parameter names, these two methods have the same signature and cannot be declared within the same class. Changing the order of parameter types does allow the method to compile, though:

Türkçe: Parameter name'leri farklı olsa da bu iki method'un signature'ı aynıdır; bu nedenle aynı class içinde birlikte bildirilemezler. Buna karşılık parameter type'larının sırasını değiştirmek farklı bir signature oluşturur ve code derlenir:

```
public class Trip {  
    public void visitZoo(String name, int waitTime) {}  
    public void visitZoo(int rainFall, String attraction) {}  
}
```

English: We cover these rules in more detail when we get to method overloading later in this chapter.

Türkçe: Bu bölümün ilerleyen kısımlarında method overloading konusuna geldiğimizde bu kuralları daha ayrıntılı olarak ele alacağız.

Exception List

Türkçe başlık: Exception Listesi

English: In Java, code can indicate that something went wrong by throwing an exception. We cover this in Chapter 11, “Exceptions and Localization.” For now, you just need to know that it is optional and where in the method declaration it goes if present. For example, `InterruptedException` is a type of `Exception`. You can list as many types of exceptions as you want in this clause, separated by commas. Here’s an example:

Türkçe: Java’da kod, bir exception fırlatarak bir şeylerin ters gittiğini gösterebilir. Bunu Chapter 11, “Exceptions and Localization” bölümünde ele alacağız. Şimdilik exception list’in isteğe bağlı olduğunu ve varsa method declaration içinde nereye yazıldığını bilmeniz yeterlidir. Örneğin `InterruptedException`, bir `Exception` type’ıdır. Bu clause içinde istediğiniz sayıda exception type’ını virgülle ayırarak listeleyebilirsiniz:

```
public class ZooMonorail {  
    public void zeroExceptions() {}  
}
```

Source page 0228

```
public void oneException() throws IllegalArgumentException {}  
public void twoExceptions() throws  
    IllegalArgumentException, InterruptedException {}  
}
```

English: While the list of exceptions is optional, it may be required by the compiler, depending on what appears inside the method body. You learn more about this, as well as how methods calling them may be required to handle these exception declarations, in Chapter 11.

Türkçe: Exception list syntax bakımından optional’dır; ancak method body’deki code’a göre compiler belirli exception’ların bildirilmesini zorunlu tutabilir. Chapter 11’de hem bu kuralları hem de caller method’ların bu exception declaration’larını nasıl handle etmesi gerektiğini öğreneceksiniz.

Method Body

Türkçe başlık: Method Body

English: The final part of a method declaration is the method body. A method body is simply a code block. It has braces that contain zero or more Java statements. We’ve spent several chapters looking at Java statements by now, so you should find it easy to figure out why these compile or don’t:

Türkçe: Method declaration’ın son bölümü method body’dir. Method body yalnızca bir code block’tur; sıfır veya daha fazla Java statement içeren süslü parantezlerden oluşur. Birkaç chapter boyunca Java statement’larını incelediğiniz için aşağıdakilerin neden derlenip derlenmediğini kolayca bulabilmelisiniz:

```
public class Bird {  
    public void fly1() {}  
    public void fly2() // DOES NOT COMPILE  
    public void fly3(int a) { int name = 5; }  
}
```

English: The `fly1()` method is a valid declaration with an empty method body. The `fly2()` method doesn't compile because it is missing the braces around the empty method body.

Türkçe: `fly1()` boş method body'ye sahip geçerli bir declaration'dır. `fly2()` ise boş method body'yi çevrelemesi gereken brace'ler bulunmadığı için derlenmez.

English: Methods are required to have a body unless they are declared abstract. We cover abstract methods in Chapter 6, "Class Design." The `fly3()` method is a valid declaration with one statement in the method body.

Türkçe: Method'lar `abstract` bildirilmedikçe bir body'ye sahip olmak zorundadır. Abstract method'ları Chapter 6, "Class Design" bölümünde ele alacağız. `fly3()`, method body içinde tek statement bulunan geçerli bir bildirimdir.

English: Congratulations! You've made it through the basics of identifying correct and incorrect method declarations. Now you can delve into more detail.

Türkçe: Tebrikler! Doğru ve yanlış method bildirimlerini tanımlamanın temellerini tamamladınız. Artık daha detaylı inceleyebilirsiniz.

Declaring Local and Instance Variables

Türkçe başlık: Local ve Instance Variable Bildirme

English: Now that we have methods, we need to talk a little bit about the variables that they can create or use. As you might recall from Chapter 1, local variables are those defined with a method or block, while instance variables are those that are defined as a member of a class.

Türkçe: Artık method'larımız olduğuna göre onların oluşturabileceği veya kullanabileceği variable'ları inceleyelim. Chapter 1'den hatırlayacağınız gibi local variable bir method veya block içinde, instance variable ise class member'ı olarak tanımlanır.

English: Let's take a look at an example:

Türkçe: Bir örneğe bakalım:

```
public class Lion {
    int hunger = 4;
    public int feedZooAnimals() {
        int snack = 10; // Local variable
```

Source page 0229

```
        if(snack > 4) {
            long dinnerTime = snack++;
            hunger--;
        }
        return snack;
    }
}
```

English: In the `Lion` class, `snack` and `dinnerTime` are local variables only accessible within their respective code blocks, while `hunger` is an instance variable and created in every object of the `Lion` class.

Türkçe: `Lion` class'ında `snack` ve `dinnerTime`, yalnızca kendi code block'ları içinde erişilebilen local variable'lardır. `hunger` ise bir instance variable'dır ve her `Lion` object'i için ayrı oluşturulur.

Editor note: Basılı kaynakta prose içindeki identifier `dinnertime` yazılmıştır. Java büyük/küçük harfe duyarlı olduğundan, yukarıdaki code ile eşleşmesi için `dinnerTime` olarak düzeltilmiştir.

English: The object or value returned by a method may be available outside the method, but the variable reference `snack` is gone. Keep this in mind while reading this chapter: all local variable references are destroyed after the block is executed, but the objects they point to may still be accessible.

Türkçe: Bir method'un döndürdüğü object veya value method dışında varlığını sürdürebilir; ancak `snack` variable reference'ı artık yoktur. Şunu akılda tutun: block çalıştıktan sonra bütün local variable reference'ları ortadan kalkar, fakat işaret ettikleri object'lere başka reference'lar üzerinden hâlâ erişilebilir olabilir.

Local Variable Modifiers

Türkçe başlık: Local Variable Modifier'ları

English: There's only one modifier that can be applied to a local variable: `final`. Easy to remember, right? When writing methods, developers may want to set a variable that does not change during the course of the method. In this code sample, trying to change the value or object these variables reference results in a compiler error:

Türkçe: Local variable'a uygulanabilen tek modifier `final`dır; hatırlaması kolay, değil mi? Developer, method execution boyunca aynı kalması gereken bir variable tanımlamak isteyebilir. Aşağıdaki code'da bu `final` variable'ların reference value'larını değiştirmeye çalışmak compiler error verir:

```
public void zooAnimalCheckup(boolean isWeekend) {
    final int rest;
    if(isWeekend) rest = 5; else rest = 20;
    System.out.print(rest);
    final var giraffe = new Animal();
    final int[] friends = new int[5];
    rest = 10; // DOES NOT COMPILE
    giraffe = new Animal(); // DOES NOT COMPILE
    friends = null; // DOES NOT COMPILE
}
```

English: As shown with the `rest` variable, we don't need to assign a value when a `final` variable is declared. The rule is only that it must be assigned a value before it can be used. We can even use `var` and `final` together. Contrast this with the following example:

Türkçe: `rest` variable'ında görüldüğü gibi `final` variable bildirildiği anda değer vermek zorunda değiliz. Tek kural, kullanılmadan önce bir değer atanmış olmasıdır. `var` ile `final` birlikte de kullanılabilir. Bunu şu örneklerle karşılaştırın:

```
public void zooAnimalCheckup(boolean isWeekend) {  
    final int rest;  
    if(isWeekend) rest = 5;  
    System.out.print(rest); // DOES NOT COMPILE  
}
```

Source page 0230

English: The rest variable might not have been assigned a value, such as if isWeekend is false.

Türkçe: Örneğin `isWeekend` değeri `false` ise `rest` variable'ına değer atanmamış olabilir.

English: Since the compiler does not allow the use of local variables that may not have been assigned a value, the code does not compile.

Türkçe: Compiler, initialize edilmemiş olabilecek local variable'ların kullanılmasına izin vermediği için code derlenmez.

English: Does using the final modifier mean we can't modify the data? Nope. The final attribute only refers to the variable reference; the contents can be freely modified (assuming the object isn't immutable).

Türkçe: `final` modifier kullanmak veriyi hiç değiştiremeyeceğimiz anlamına mı gelir? Hayır. `final` yalnızca variable reference'ı kısıtlar; object immutable değilse içeriği serbestçe değiştirilebilir.

```
public void zooAnimalCheckup() {  
    final int rest = 5;  
    final Animal giraffe = new Animal();  
    final int[] friends = new int[5];  
    giraffe.setName("George");  
    friends[2] = 2;  
}
```

English: The rest variable is a primitive, so it's just a value that can't be modified. On the other hand, the contents of the giraffe and friends variables can be freely modified, provided the variables aren't reassigned.

Türkçe: `rest` bir primitive variable olduğundan, değiştirilemeyen tek bir value taşır. Buna karşılık `giraffe` ve `friends` variable'larının işaret ettiği object/array içeriği, bu variable'lar yeniden atanmadığı sürece değiştirilebilir.

English: While it might not seem obvious, marking a local variable final is often a good practice. For example, you may have a complex method in which a variable is referenced dozens of times. It would be really bad if someone came in and reassigned the variable in the middle of the method. Using the final attribute is like sending a message to other developers to leave the variable alone!

Türkçe: İlk bakışta gerekli görünmese de local variable'ı `final` olarak işaretlemek genellikle iyi bir uygulamadır. Karmaşık bir method'da aynı variable'a onlarca kez başvurulabilir; method'un ortasında yanlışlıkla yeniden atama yapılması ciddi sorun yaratır. `final`, diğer developer'lara bu variable'ı yeniden atamamaları gerektiğini anlatır.

Effectively Final Variables

Türkçe başlık: Effectively Final Variable'lar

English: An effectively final local variable is one that is not modified after it is assigned. This means that the value of a variable doesn't change after it is set, regardless of whether it is explicitly marked as final. If you aren't sure whether a local variable is effectively final, just add the final keyword. If the code still compiles, the variable is effectively final.

Türkçe: Effectively final local variable, ilk değer atamasından sonra değiştirilmeyen variable'dır. Başka bir deyişle, açıkça `final` yazılmış olsun veya olmasın value bir kez ayarlandıktan sonra değişmez. Bir local variable'ın effectively final olup olmadığından emin değilseniz declaration'a `final` ekleyin; kod hâlâ derleniyorsa variable effectively final'dır.

English: Given this definition, which of the following variables are effectively final?

Türkçe: Bu tanıma göre aşağıdaki variable'lardan hangileri effectively final'dır?

```
11: public String zooFriends() {  
12:   String name = "Harry the Hippo";  
13:   var size = 10;  
14:   boolean wet;  
15:   if(size > 100) size++;  
16:   name.substring(0);  
17:   wet = true;  
18:   return name;  
19: }
```

Source page 0231

English: Remember, a quick test of effectively final is to just add final to the variable declaration and see if it still compiles. In this example, name and wet are effectively final and can be updated with the final modifier, but not size. The name variable is assigned a value on line 12 and not reassigned. Line 16 creates a value that is never used. Remember from Chapter 4, "Core APIs," that strings are immutable. The size variable is not effectively final because it could be incremented on line 15. The wet variable is assigned a value only once and not modified afterward.

Türkçe: Effectively final için hızlı test, variable declaration'a `final` ekleyip kodun hâlâ derlenip derlenmediğine bakmaktır. Bu örnekte `name` ve `wet` effectively final'dır ve `final` modifier ile bildirilebilir; `size` ise bildirilemez. `name` 12. satırda atanır ve yeniden atanmaz. 16. satır, sonucu hiç kullanılmayan yeni bir `String` value üretir; Chapter 4'ten hatırlayın, `String` immutable'dır. `size` 15. satırda artırılabilirdi için effectively final değildir. `wet` yalnızca bir kez atanır ve sonrasında değiştirilmez.

Effective Final Parameters

Türkçe başlık: Effectively Final Parameter'lar

English: Recall from Chapter 1 that method and constructor parameters are local variables that have been pre-initialized. In the context of local variables, the same rules around final and effectively final apply. This is especially important in Chapter 7 and Chapter 8, “Lambdas and Functional Interfaces,” since local classes and lambda expressions declared within a method can only reference local variables that are final or effectively final.

Türkçe: Chapter 1'den method ve constructor parameter'larının önceden initialize edilmiş local variable'lar olduğunu hatırlayın. Local variable bağlamında `final` ve effectively final için aynı kurallar geçerlidir. Bu konu özellikle Chapter 7 ile Chapter 8, “Lambdas and Functional Interfaces” için önemlidir; çünkü bir method içinde bildirilen local class'lar ve lambda expression'lar yalnızca `final` veya effectively final local variable'lara başvurabilir.

Instance Variable Modifiers

Türkçe başlık: Instance Variable Modifier'ları

English: Like methods, instance variables can use access modifiers, such as private, package, protected, and public. Remember, package access is indicated by the lack of any modifiers. We cover each of the different access modifiers shortly in this chapter. Instance variables can also use optional specifiers, described in Table 5.3.

Türkçe: Method'lar gibi instance variable'lar da `private`, `package`, `protected` ve `public` access modifier'larını kullanabilir. Package access, hiçbir access modifier yazılmamasıyla gösterilir. Farklı access modifier'ları bu bölümde birazdan inceleyeceğiz. Instance variable'lar ayrıca Tablo 5.3'teki optional specifier'ları kullanabilir.

English: TABLE 5.3 Optional specifiers for instance variables

Türkçe: TABLO 5.3 Instance variable'lar için optional specifier'lar

English: Modifier · Description · Chapter Covered

Türkçe: Modifier · Açıklama · Ele alındığı chapter

English: `final` · Specifies that the instance variable must be initialized with each instance of the class exactly once · Chapter 5

Türkçe: `final` · Instance variable'ın class'ın her instance'ı için tam bir kez initialize edilmesini zorunlu kılar · Chapter 5

English: `volatile` · Instructs the JVM that the value in this variable may be modified by other threads · Chapter 13

Türkçe: `volatile` · Bu variable'daki value'nun başka thread'ler tarafından değiştirilebileceğini JVM'e bildirir · Chapter 13

English: `transient` · Used to indicate that an instance variable should not be serialized with the class · Chapter 14

Türkçe: `transient` · Instance variable'ın class ile birlikte serialize edilmemesi gerektiğini belirtir · Chapter 14

English: Looks like we only need to discuss `final` in this chapter! If an instance variable is marked `final`, then it must be assigned a value when it is declared or when the object is

Türkçe: Görünüşe göre bu bölümde yalnızca `final` konusunu ele almamız gerekiyor! Bir instance variable `final` olarak işaretlenmişse declaration sırasında ya da object...

Source page 0232

English: instantiated. Like a local `final` variable, it cannot be assigned a value more than once, though. The following `PolarBear` class demonstrates these properties:

Türkçe: ...instantiate edilirken bir value almalıdır. Local `final` variable gibi buna da birden fazla kez value atanamaz. Aşağıdaki `PolarBear` class'ı bu özellikleri gösterir:

```
public class PolarBear {  
    final int age = 10;  
    final int fishEaten;  
    final String name;  
    { fishEaten = 10; }  
    public PolarBear() {  
        name = "Robert";  
    }  
}
```

English: The age variable is given a value when it is declared, while the fishEaten variable is assigned a value in an instance initializer. The name variable is given a value in the no-argument constructor. Failing to initialize an instance variable (or assigning a value more than once) will lead to a compiler error. We talk about final variable initialization in more detail when we cover constructors in the next chapter.

Türkçe: `age` declaration sırasında, `fishEaten` instance initializer içinde, `name` ise no-argument constructor'da değer alır. Bir instance variable'ı initialize etmemek veya ona birden fazla kez değer atamak compiler error'a yol açar. `final` variable initialization'ını sonraki chapter'da constructor'larla birlikte daha ayrıntılı ele alacağız.

English: In Chapter 1, we show that instance variables receive default values based on their type when not set. For example, `int` receives a default value of 0, while an object reference receives a default value of null. The compiler does not apply a default value to final variables, though. A final instance or final static variable must receive a value when it is declared or as part of initialization.

Türkçe: Chapter 1'de instance variable'ların açıkça ayarlanmadığında type'larına göre default value aldığını gördük. Örneğin `int` için default 0, object reference için `null` olur. Ancak compiler `final` variable'lara default value atamaz. `final` instance veya `final static` variable, declaration sırasında ya da initialization sürecinde bir value almak zorundadır.

Working with Varargs

Türkçe başlık: Varargs'la çalışmak

English: As mentioned in Chapter 4, a method may use a varargs parameter (variable argument) as if it is an array. Creating a method with a varargs parameter is a bit more complicated. In fact, calling such a method may not use an array at all.

Türkçe: Chapter 4'te belirtildiği gibi bir method, varargs parameter'ını (variable argument) array gibi kullanabilir. Varargs parameter'lı bir method oluşturmak biraz daha karmaşıktır; üstelik böyle bir method çağrılırken doğrudan array kullanılması şart değildir.

Creating Methods with Varargs

Türkçe başlık: Varargs ile Method Oluşturma

English: There are a number of important rules for creating a method with a varargs parameter.

Türkçe: Varargs parameter'lı method oluşturmak için birkaç önemli kural vardır.

English: Rules for Creating a Method with a Varargs Parameter

Türkçe: Varargs Parameter'lı Method Oluşturma Kuralları

English: 1. A method can have at most one varargs parameter.

Türkçe: 1. Bir method en fazla bir varargs parameter içerebilir.

English: 2. If a method contains a varargs parameter, it must be the last parameter in the list.

Türkçe: 2. Bir method varargs parameter içeriyorsa bu parameter list'in son ögesi olmak zorundadır.

Source page 0233

English: Given these rules, can you identify why each of these does or doesn't compile? (Yes, there is a lot of practice in this chapter. You have to be really good at identifying valid and invalid methods for the exam.)

Türkçe: Bu kurallara göre her bildirimin neden derlenip derlenmediğini belirleyebilir misiniz? (Evet, bu bölümde bolca pratik var. Sınav için geçerli ve geçersiz method'ları çok iyi ayırt etmelisiniz.)

```
public class VisitAttractions {
    public void walk1(int... steps) {}
    public void walk2(int start, int... steps) {}
    public void walk3(int... steps, int start) {} // DOES NOT COMPILE
    public void walk4(int... start, int... steps) {} // DOES NOT COMPILE
}
```

English: The `walk1()` method is a valid declaration with one varargs parameter. The `walk2()` method is a valid declaration with one `int` parameter and one varargs parameter. The `walk3()` and `walk4()` methods do not compile because they have a varargs parameter in a position that is not the last one.

Türkçe: `walk1()` bir varargs parameter'a sahip geçerli declaration'dır. `walk2()` bir `int` parameter ve bir varargs parameter içeren geçerli declaration'dır. `walk3()` ile `walk4()`, varargs parameter list'in son sırasında olmadığı için derlenmez.

Calling Methods with Varargs

Türkçe başlık: Varargs ile Method'ları Çağırma

English: When calling a method with a varargs parameter, you have a choice. You can pass in an array, or you can list the elements of the array and let Java create it for you. Given our previous `walk1()` method, which takes a varargs parameter, we can call it one of two ways:

Türkçe: Varargs parameter'lı bir method'u çağırırken iki seçeneğiniz vardır: bir array geçebilir ya da array elemanlarını tek tek yazarak array'i Java'nın oluşturmasını sağlayabilirsiniz. Önceki `walk1()` method'u şu iki biçimde çağırılabilir:

```
// Pass an array
int[] data = new int[] {1, 2, 3};
walk1(data);
// Pass a list of values
walk1(1,2,3);
```

English: Regardless of which one you use to call the method, the method will receive an array containing the elements. We can reinforce this with the following example:

Türkçe: Hangi çağrı biçimini kullanırsanız kullanın method, elemanları içeren bir array alır. Şu örnek bunu pekiştirir:

```
public void walk1(int... steps) {
    int[] step2 = steps; // Not necessary, but shows steps is of type int[]
    System.out.print(step2.length);
}
```

English: You can even omit the varargs values in the method call, and Java will create an array of length zero for you.

Türkçe: Method call'da varargs argument'larını tamamen atlayabilirsiniz; Java length'i 0 olan bir array oluşturur.

```
walk1();
```

Source page 0234

Accessing Elements of a Vararg

Türkçe başlık: Vararg Element'lerine Erişim

English: Accessing a varargs parameter is just like accessing an array. It uses array indexing. Here's an example:

Türkçe: Varargs parameter'a erişmek array'e erişmekle aynıdır; array indexing kullanılır. Örneğin:

```
16: public static void run(int... steps) {
17:     System.out.print(steps[1]);
18: }
19: public static void main(String[] args) {
20:     run(11, 77); // 77
21: }
```

English: Line 20 calls a varargs method with two parameters. When the method is called, it sees an array of size 2. Since indexes are zero-based, 77 is printed.

Türkçe: 20. satır varargs method'a iki argument geçirir. Method length'i 2 olan bir array alır. Index'ler zero-based olduğu için `steps[1]` value'su olan `77` yazdırılır.

Using Varargs with Other Method Parameters

Türkçe başlık: Varargs'ı Diğer Method Parameter'larıyla Kullanma

English: Finally! You get to do something other than identify whether method declarations are valid.

Türkçe: Sonunda! Artık yalnızca method declaration'ların geçerli olup olmadığını belirlemek dışında bir şey yapacaksınız.

English: Instead, you get to look at method calls. Can you figure out why each method call outputs what it does? For now, feel free to ignore the static modifier in the `walkDog()` method declaration; we cover that later in the chapter.

Türkçe: Şimdi method call'ları inceleyeceksiniz. Her çağrının neden ilgili çıktıyı ürettiğini bulabilir misiniz? Şimdilik `walkDog()` declaration'ındaki `static` modifier'ı göz ardı edebilirsiniz; bunu chapter'ın ilerleyen kısmında ele alacağız.

```
1: public class DogWalker {
2:     public static void walkDog(int start, int... steps) {
3:         System.out.println(steps.length);
4:     }
5:     public static void main(String[] args) {
6:         walkDog(1); // 0
7:         walkDog(1, 2); // 1
8:         walkDog(1, 2, 3); // 2
9:         walkDog(1, new int[] {4, 5}); // 2
10:    }
```

English: Line 6 passes 1 as start but nothing else. This means Java creates an array of length 0 for steps. Line 7 passes 1 as start and one more value. Java converts this one value to an array of length 1. Line 8 passes 1 as start and two more values. Java converts these two values to an array of length 2. Line 9 passes 1 as start and an array of length 2 directly as steps.

Türkçe: 6. satır `start` için 1 geçirir, başka value geçirmez; Java bu nedenle `steps` için length'i 0 olan bir array oluşturur. 7. satır `start` için 1 ve bir ek value geçirir; Java bu value'yu length'i 1 olan array'e dönüştürür. 8. satırdaki iki ek value length'i 2 olan array'e dönüşür. 9. satır ise length'i 2 olan array'i doğrudan `steps` olarak geçirir.

English: You've seen that Java will create an empty array if no parameters are passed for a vararg.

Türkçe: Vararg için hiçbir argument geçirilmezse Java'nın boş bir array oluşturacağını gördünüz.

English: However, it is still possible to pass null explicitly. The following snippet does compile:

Türkçe: Yine de açıkça `null` geçirmek mümkündür. Aşağıdaki kod parçası derlenir:

```
walkDog(1, null); // Triggers NullPointerException in walkDog()
```

English: Since null isn't an int, Java treats it as an array reference that happens to be null. It just passes on the null array object to walkDog(). Then the walkDog() method throws an exception because it tries to determine the length of null.

Türkçe: `null` bir `int` olmadığı için Java onu değeri `null` olan bir array reference olarak yorumlar ve bu `null` array object'ini `walkDog()` method'una geçirir. Ardından `walkDog()` method'u `null` değerinin length'ini belirlemeye çalıştığı için exception fırlatır.

Source page 0235

Applying Access Modifiers

Türkçe başlık: Access Modifier'ları Uygulama

English: You already saw that there are four access modifiers: private, package, protected, and public access. We are going to discuss them in order from most restrictive to least restrictive:

Türkçe: Dört access düzeyini gördünüz: `private`, package, `protected` ve `public`. Şimdi bunları en kısıtlayıcıdan en az kısıtlayıcıya doğru inceleyeceğiz:

English: • `private`: Only accessible within the same class.

Türkçe: • `private`: Yalnızca aynı class içinden erişilebilir.

English: • Package access: `private` plus other members of the same package. Sometimes referred to as package-private or default access.

Türkçe: • Package access: `private` erişime ek olarak aynı package içindeki diğer member'lardan da erişim sağlar. Package-private veya default access olarak da adlandırılır.

English: • `protected`: Package access plus access within subclasses.

Türkçe: • `protected`: Package access'e ek olarak subclass'lardan erişim sağlar.

English: • `public`: `protected` plus classes in the other packages.

Türkçe: • `public`: `protected` erişime ek olarak diğer package'lerdeki class'lara da erişim sağlar.

English: We will explore the impact of these four levels of access on members of a class.

Türkçe: Bu dört erişim düzeyinin bir class'ın üyeleri üzerindeki etkisini araştıracağız.

Private Access

Türkçe başlık: Private Access

English: Let's start with private access, which is the simplest. Only code in the same class can call private methods or access private fields.

Türkçe: En basit düzey olan `private` access ile başlayalım. Yalnızca aynı class içindeki kod `private` method'ları çağırabilir veya `private` field'lara erişebilir.

English: First, take a look at Figure 5.2. It shows the classes you'll use to explore private and package access. The big boxes are the names of the packages. The smaller boxes inside them are the classes in each package. You can refer back to this figure if you want to quickly see how the classes relate.

Türkçe: Önce Şekil 5.2'ye bakın. Bu şekil, `private` ve package access'i incelerken kullanacağınız class'ları gösterir. Büyük kutular package adlarını, içlerindeki küçük kutular ise her package'teki class'ları temsil eder. Class'lar arasındaki ilişkiyi hızla hatırlamak için bu şekle dönebilirsiniz.

English: FIGURE 5.2 Classes used to show private and package access

Türkçe: ŞEKİL 5.2 `private` ve package access'i göstermek için kullanılan class'lar

English: `pond.duck`

Türkçe: `pond.duck`

English: `FatherDuck`

Türkçe: `FatherDuck`

English: `MotherDuck`

Türkçe: `MotherDuck`

English: `BadDuckling`

Türkçe: `BadDuckling`

English: `GoodDuckling`

Türkçe: `GoodDuckling`

English: `pond.swan`

Türkçe: `pond.swan`

English: BadCygnet

Türkçe: BadCygnet

English: This is perfectly legal code because everything is one class:

Türkçe: Bu code tamamen geçerlidir; çünkü her şey tek bir class içindedir:

```
1: package pond.duck;
2: public class FatherDuck {
```

Source page 0236

```
3: private String noise = "quack";
4: private void quack() {
5: System.out.print(noise); // private access is ok
6: }
7: }
```

English: So far, so good. FatherDuck declares a private method quack() and uses private instance variable noise on line 5.

Türkçe: Buraya kadar sorun yok. `FatherDuck`, `private` method `quack()`'i bildirir ve 5. satırda `private` instance variable `noise`'u kullanır.

English: Now we add another class:

Türkçe: Şimdi başka bir class ekliyoruz:

```
1: package pond.duck;
2: public class BadDuckling {
3: public void makeNoise() {
4: var duck = new FatherDuck();
5: duck.quack(); // DOES NOT COMPILE
6: System.out.print(duck.noise); // DOES NOT COMPILE
7: }
8: }
```

English: BadDuckling is trying to access an instance variable and a method it has no business touching. On line 5, it tries to access a private method in another class. On line 6, it tries to access a private instance variable in another class. Both generate compiler errors.

Türkçe: `BadDuckling`, erişme hakkı olmayan bir instance variable ve method'a ulaşmaya çalışır. 5. satır başka bir class'taki `private` method'a, 6. satır ise başka bir class'taki `private` instance variable'a erişir. İki satır da compiler error üretir.

English: Bad duckling!

Türkçe: Kötü ördek yavrusu!

English: Our bad duckling is only a few days old and doesn't know better yet. Luckily, you know that accessing private members of other classes is not allowed, and you need to use a different type of access.

Türkçe: Kötü ördek yavrumuz yalnızca birkaç günlük ve henüz doğrusunu bilmiyor. Neyse ki siz, başka class'ların `private` member'larına erişilemeyeceğini ve farklı bir access düzeyi kullanılması gerektiğini biliyorsunuz.

English: In the previous example, FatherDuck and BadDuckling are in separate files, but what if they were declared in the same file? Even then, the code would still not compile as Java prevents access outside the class.

Türkçe: Önceki örnekte, FatherDuck ve BadDuckling ayrı dosyalardadır, peki ya aynı dosyada bildirilmişlerse? O zaman bile, Java class dışına erişimi engellediğinden kod yine de derlenmez.

Package Access

Türkçe başlık: Package Access

English: Luckily, MotherDuck is more accommodating about what her ducklings can do. She allows classes in the same package to access her members. When there is no access modifier, Java assumes package access.

Türkçe: Neyse ki `MotherDuck` daha hoşgörülüdür ve aynı package içindeki class'ların onun member'larına erişmesine izin verir. Hiç access modifier yazılmadığında Java package access varsayar.

```
package pond.duck;
public class MotherDuck {
    String noise = "quack";
    void quack() {
        System.out.print(noise); // package access is ok
    }
}
```

Source page 0237

English: MotherDuck can refer to noise and call quack(). After all, members in the same class are certainly in the same package. The big difference is that MotherDuck lets other classes in the same package access members, whereas FatherDuck doesn't (due to being private).

Türkçe: `MotherDuck`, `noise`'a erişip `quack()`'i çağırabilir; aynı class'taki üyeler zaten aynı package içindedir. Asıl fark, `MotherDuck`'ın aynı package'teki diğer class'lara da erişim vermesi, `FatherDuck`'ın ise üyeleri `private` olduğu için vermemesidir.

English: GoodDuckling has a much better experience than BadDuckling:

Türkçe: `GoodDuckling`'in deneyimi `BadDuckling`'inkinden çok daha iyidir:

```
package pond.duck;
public class GoodDuckling {
    public void makeNoise() {
        var duck = new MotherDuck();
        duck.quack(); // package access is ok
        System.out.print(duck.noise); // package access is ok
    }
}
```

English: GoodDuckling succeeds in learning to quack() and make noise by copying its mother.

Türkçe: `GoodDuckling`, annesini taklit ederek `quack()` etmeyi ve ses çıkarmayı başarır.

English: Notice that all the classes covered so far are in the same package, pond.duck. This allows package access to work.

Türkçe: Şimdiye kadarki bütün class'ların aynı `pond.duck` package'inde olduğuna dikkat edin. Package access bu nedenle çalışır.

English: In this same pond, a swan just gave birth to a baby swan. A baby swan is called a cygnet.

Türkçe: Aynı gölette bir kuğunun yavrusu olmuştur. Yavru kuğuya İngilizcede "cygnet" denir.

English: The cygnet sees the ducklings learning to quack and decides to learn from MotherDuck as well.

Türkçe: Kuğu yavrusu, ördek yavrularının vaklamayı öğrendiğini görür ve MotherDuck'tan da öğrenmeye karar verir.

```
package pond.swan;
import pond.duck.MotherDuck; // import another package
public class BadCygnet {
    public void makeNoise() {
        var duck = new MotherDuck();
        duck.quack(); // DOES NOT COMPILE
        System.out.print(duck.noise); // DOES NOT COMPILE
    }
}
```

English: Oh, no! MotherDuck only allows lessons to other ducks by restricting access to the pond.duck package. Poor little BadCygnet is in the pond.swan package, and the code doesn't compile. Remember that when there is no access modifier on a member, only classes in the same package can access the member.

Türkçe: Olmadı! MotherDuck, member access'ini pond.duck package'ile sınırlamıştır. Zavallı BadCygnet ise pond.swan package'indedir; bu yüzden kod derlenmez. Bir member'da access modifier yoksa o member'a yalnızca aynı package'teki class'ların erişebileceğini unutmayın.

Protected Access

Türkçe başlık: Protected Access

English: Protected access allows everything that package access does, and more. The protected access modifier adds the ability to access members of a parent class. We cover creating subclasses in depth in Chapter 6. For now, we cover the simplest possible use of a subclass. In the following example, the “child” ClownFish class is a subclass of the “parent” Fish class, using the extends keyword to connect them:

Türkçe: protected access, package access'in sağladığı her şeyi ve fazlasını sunar: parent class üyelerine erişme yeteneği ekler. Subclass oluşturmayı Chapter 6'da ayrıntılı inceleyeceğiz. Şimdilik en basit kullanıma bakalım. Aşağıdaki “child” ClownFish, extends keyword'üyle “parent” Fish class'ına bağlanan bir subclass'tır:

```
public class Fish {}
public class ClownFish extends Fish {}
```

Source page 0238

English: By extending a class, the subclass gains access to all protected and public members of the parent class, as if they were declared in the subclass. If the two classes are in the same package, then the subclass also gains access to all package members.

Türkçe: Bir class'ı extend eden subclass, parent class'ın bütün protected ve public member'larına sanki kendi içinde bildirilmişler gibi erişir. İki class aynı package içindeyse subclass bütün package-access member'lara da erişir.

English: Figure 5.3 shows the many classes we create in this section. There are a number of classes and packages, so don't worry about keeping them all in your head. Just check back with this figure as you go.

Türkçe: Şekil 5.3, bu bölümde oluşturacağımız class'ları gösterir. Class ve package sayısı fazla olduğu için hepsini ezberlemeye çalışmayın; ilerlerken gerektiğinde şekle yeniden bakın.

English: FIGURE 5.3 Classes used to show protected access

Türkçe: ŞEKİL 5.3 `protected` access'i göstermek için kullanılan class'lar

English: `pond.shore: Bird, BirdWatcher`

Türkçe: `pond.shore: Bird, BirdWatcher`

English: `pond.goose: Gosling (extends Bird), Goose (extends Bird)`

Türkçe: `pond.goose: Gosling (extends Bird), Goose (extends Bird)`

English: `pond.inland: BirdWatcherFromAfar`

Türkçe: `pond.inland: BirdWatcherFromAfar`

English: `pond.swan: Swan (extends Bird)`

Türkçe: `pond.swan: Swan (extends Bird)`

English: `pond.duck: GooseWatcher`

Türkçe: `pond.duck: GooseWatcher`

English: First, create a `Bird` class and give protected access to its members:

Türkçe: Önce bir `Bird` class'ı oluşturun ve member'larına `protected` access verin:

```
package pond.shore;
public class Bird {
    protected String text = "floating";
    protected void floatInWater() {
        System.out.print(text); // protected access is ok
    }
}
```

English: Next, we create a subclass:

Türkçe: Ardından bir subclass oluşturuyoruz:

```
package pond.goose;
import pond.shore.Bird;
public class Gosling extends Bird { // Different package than Bird
    // Gosling is a subclass of Bird
}
```

Source page 0239

```
public void swim() {  
    floatInWater(); // protected access is ok  
    System.out.print(text); // protected access is ok  
}  
public static void main(String[] args) {  
    new Gosling().swim();  
}  
}
```

English: This is a simple subclass. It extends the Bird class. Extending means creating a subclass that has access to any protected or public members of the parent class. Running this program prints floating twice: once from calling floatInWater(), and once from the print statement in swim(). Since Gosling is a subclass of Bird, it can access these members even though it is in a different package.

Türkçe: Bu basit bir subclass'tır ve Bird class'ını extend eder. Extending, parent class'ın protected veya public member'larına erişebilen bir subclass oluşturmak demektir. Program floating değerini iki kez yazdırır: biri floatInWater() çağrısından, diğeri swim() içindeki print statement'tan gelir. Gosling, Bird subclass'ı olduğu için farklı bir package'te olsa bile bu member'lara erişebilir.

English: Remember that protected also gives us access to everything that package access does. This means a class in the same package as Bird can access its protected members.

Türkçe: protected access'in package access kapsamını da içerdiğini unutmayın. Dolayısıyla Bird ile aynı package'teki bir class, onun protected member'larına erişebilir.

```
package pond.shore; // Same package as Bird  
public class BirdWatcher {  
    public void watchBird() {  
        Bird bird = new Bird();  
        bird.floatInWater(); // protected access is ok  
        System.out.print(bird.text); // protected access is ok  
    }  
}
```

English: Since Bird and BirdWatcher are in the same package, BirdWatcher can access package members of the bird variable. The definition of protected allows access to subclasses and classes in the same package. This example uses the same package part of that definition.

Türkçe: Bird ve BirdWatcher aynı package'te olduğu için BirdWatcher, bird variable'ının protected member'larına erişebilir. protected tanımı hem subclass'lara hem aynı package'teki class'lara erişim verir; bu örnek ikinci kısmı kullanır.

English: Now let's try the same thing from a different package:

Türkçe: Şimdi aynı şeyi farklı bir package'ten deneyelim:

```
package pond.inland; // Different package than Bird
import pond.shore.Bird;
public class BirdWatcherFromAfar { // Not a subclass of Bird
    public void watchBird() {
        Bird bird = new Bird();
        bird.floatInWater(); // DOES NOT COMPILE
        System.out.print(bird.text); // DOES NOT COMPILE
    }
}
```

English: BirdWatcherFromAfar is not in the same package as Bird, and it doesn't inherit from Bird. This means it is not allowed to access protected members of Bird.

Türkçe: BirdWatcherFromAfar, Bird ile aynı package'te değildir ve Bird'den inherit etmez. Bu nedenle Bird'ün protected member'larına erişemez.

English: Got that? Subclasses and classes in the same package are the only ones allowed to access protected members.

Türkçe: Anlaşıldı mı? protected member'lara yalnızca subclass'lar ve aynı package'teki class'lar erişebilir.

Source page 0240

English: There is one gotcha for protected access. Consider this class:

Türkçe: protected access'in bir önemli tuzağı vardır. Şu class'ı inceleyin:

```
1: package pond.swan; // Different package than Bird
2: import pond.shore.Bird;
3: public class Swan extends Bird { // Swan is a subclass of Bird
4:     public void swim() {
5:         floatInWater(); // protected access is ok
6:         System.out.print(text); // protected access is ok
7:     }
8:     public void helpOtherSwanSwim() {
9:         Swan other = new Swan();
10:        other.floatInWater(); // subclass access to superclass
11:        System.out.print(other.text); // subclass access to superclass
12:    }
13:    public void helpOtherBirdSwim() {
14:        Bird other = new Bird();
15:        other.floatInWater(); // DOES NOT COMPILE
16:        System.out.print(other.text); // DOES NOT COMPILE
17:    }
18: }
```

English: Take a deep breath. This is interesting. Swan is not in the same package as Bird but does extend it— which implies it has access to the protected members of Bird since it is a subclass. And it does. Lines 5 and 6 refer to protected members via inheriting them.

Türkçe: Derin bir nefes alın; burası ilginçtir. Swan, Bird ile aynı package'te değildir ama Bird'i extend eder. Bu nedenle bir subclass olarak Bird'ün protected member'larına erişebilir. 5. ve 6. satırlar bu member'lara inheritance yoluyla erişir.

English: Lines 10 and 11 also successfully use protected members of Bird. This is allowed because these lines refer to a Swan object. Swan inherits from Bird, so this is okay. It is sort of a two-phase check. The Swan class is allowed to use protected members of Bird, and we are referring to a Swan object. Granted, it is a Swan object created on line 9 rather than an inherited one, but it is still a Swan object.

Türkçe: 10. ve 11. satırlar da Bird'ün protected member'larını başarıyla kullanır; çünkü bu kez member'lara bir Swan object'i üzerinden erişilir. Denetim iki aşamalıdır: Swan class'ı Bird'ün protected member'larını kullanabilir ve receiver reference'ın type'ı da Swan'dır. Object 9. satırda ayrıca oluşturulmuş olsa da yine bir Swan object'idir.

English: Lines 15 and 16 do not compile. Wait a minute. They are almost exactly the same as lines 10 and 11! There's one key difference. This time a Bird reference is used rather than inheritance. It is created on line 14. Bird is in a different package, and this code isn't inheriting from Bird, so it doesn't get to use protected members. Say what, now? We just got through saying repeatedly that Swan inherits from Bird. And it does. However, the variable reference isn't a Swan. The code just happens to be in the Swan class.

Türkçe: 15. ve 16. satırlar derlenmez; 10. ve 11. satırlardan kritik bir farkları vardır. Bu kez member'lara 14. satırda oluşturulan Bird reference'ı üzerinden erişilir. Bird başka bir package'tedir ve receiver'in reference type'ı Swan değildir. Kodun Swan class'ında bulunması tek başına yetmez.

English: It's okay to be confused. This is arguably one of the most confusing points on the exam.

Türkçe: Kafanın karışması sorun değil. Bu muhtemelen sınavdaki en kafa karıştırıcı noktalardan biridir.

English: Looking at it a different way, the protected rules apply under two scenarios:

Türkçe: Başka bir açıdan bakarsak protected kuralları iki senaryoda uygulanır:

English: • A member is used without referring to a variable. This is the case on lines 5 and 6. In this case, we are taking advantage of inheritance, and protected access is allowed.

Türkçe: • Member bir variable üzerinden başvurulmadan kullanılır (5. ve 6. satırlar). Burada inheritance devrededir ve protected access geçerlidir.

English: • A member is used through a variable. This is the case on lines 10, 11, 15, and 16. In this case, the rules for the reference type of the variable are what matter. If it is a subclass, protected access is allowed. This works for references to the same class or a subclass.

Türkçe: • Member bir variable üzerinden kullanılır (10, 11, 15 ve 16. satırlar). Bu durumda variable'ın reference type'ı önemlidir. Aynı subclass type'ı veya onun bir subclass'ıysa protected access'e izin verilir.

Source page 0241

English: We're going to try this again to make sure you understand what is going on. Can you figure out why these examples don't compile?

Türkçe: Neler olup bittiğini anladığınızdan emin olmak için bunu tekrar deneyeceğiz. Bu örneklerin neden derlenmediğini anlayabiliyor musunuz?

```
package pond.goose;
import pond.shore.Bird;
public class Goose extends Bird {
    public void helpGooseSwim() {
        Goose other = new Goose();
        other.floatInWater();
        System.out.print(other.text);
    }
    public void helpOtherGooseSwim() {
        Bird other = new Goose();
        other.floatInWater(); // DOES NOT COMPILE
        System.out.print(other.text); // DOES NOT COMPILE
    }
}
```

English: The first method is fine. In fact, it is equivalent to the Swan example. Goose extends Bird. Since we are in the Goose subclass and referring to a Goose reference, it can access protected members. The second method is a problem. Although the object happens to be a Goose, it is stored in a Bird reference. We are not allowed to refer to members of the Bird class since we are not in the same package and the reference type of other is not a subclass of Goose.

Türkçe: İlk method geçerlidir ve Swan örneğine denktir. Goose, Bird'i extend eder; kod Goose subclass'ı içindedir ve receiver reference'ın type'ı Goose olduğu için protected member'lara erişilebilir. İkinci method ise sorunludur. Runtime object bir Goose olsa da other variable'ının reference type'ı Bird'dür. Kod Bird ile aynı package'te değildir ve Bird, Goose'un subclass'ı olmadığından bu reference üzerinden Bird member'larına erişilemez.

English: What about this one?

Türkçe: Buna ne dersiniz?

```
package pond.duck;
import pond.goose.Goose;
public class GooseWatcher {
    public void watch() {
        Goose goose = new Goose();
        goose.floatInWater(); // DOES NOT COMPILE
    }
}
```

English: This code doesn't compile because we are not in the goose object. The floatInWater() method is declared in Bird. GooseWatcher is not in the same package as Bird, nor does it extend Bird. Goose extends Bird. That only lets Goose refer to floatInWater(), not callers of Goose.

Türkçe: Bu kod derlenmez; çünkü erişim goose object'inin içinden yapılmıyor. floatInWater() Bird içinde bildirilmiştir. GooseWatcher, Bird ile aynı package'te değildir ve Bird'ü extend etmez. Goose'un Bird'ü extend etmesi yalnızca Goose class'ına erişim hakkı verir; Goose çağıranlarına vermez.

English: If this is still puzzling, try it. Type in the code and try to make it compile. Then reread this section. Don't worry— it wasn't obvious to us the first time either!

Türkçe: Bu hâlâ kafa karıştırıcıysa deneyin. Kodu yazıp derlemeye çalışın, ardından bu bölümü yeniden okuyun. Endişelenmeyin; ilk seferinde bizim için de pek açık değildi!

Source page 0242

Public Access

Türkçe başlık: Public Access

English: Protected access was a tough concept. Luckily, the last type of access modifier is easy: public means anyone can access the member from anywhere.

Türkçe: `protected` access zordu. Neyse ki son access düzeyi kolaydır: `public`, herkesin member'a her yerden erişebilmesi demektir.

English: The Java module system redefines “anywhere,” and it becomes possible to restrict access to public code outside a module. We cover this in more detail in Chapter 12, “Modules.” When given code samples, you can assume they are in the same module unless explicitly stated otherwise.

Türkçe: Java module system, “her yerden” ifadesinin kapsamını yeniden tanımlar; bir module dışından `public` code'a erişim kısıtlanabilir. Bunu Chapter 12, “Modules” bölümünde ayrıntılı inceleyeceğiz. Aksi açıkça belirtilmedikçe verilen code sample'ların aynı module'de olduğunu varsayabilirsiniz.

English: Let's create a class that has public members:

Türkçe: Public üyeleri olan bir class oluşturalım:

```
package pond.duck;
public class DuckTeacher {
    public String name = "helpful";
    public void swim() {
        System.out.print(name); // public access is ok
    }
}
```

English: DuckTeacher allows access to any class that wants it. Now we can try it:

Türkçe: `DuckTeacher`, erişmek isteyen her class'a izin verir. Şimdi bunu deneyebiliriz:

```
package pond.goose;
import pond.duck.DuckTeacher;
public class LostDuckling {
    public void swim() {
        var teacher = new DuckTeacher();
        teacher.swim(); // allowed
        System.out.print("Thanks" + teacher.name); // allowed
    }
}
```

English: LostDuckling is able to refer to swim() and name on DuckTeacher because they are public. The story has a happy ending. LostDuckling has learned to swim and can find its parents— all because DuckTeacher made members public.

Türkçe: `swim()` ve `name` `public` olduğu için `LostDuckling`, `DuckTeacher` üzerindeki bu member'lara erişebilir. Hikâye mutlu biter: `DuckTeacher` member'ları `public` yaptığı için `LostDuckling` yüzmeyi öğrenip ailesini bulabilir.

Reviewing Access Modifiers

Türkçe başlık: Access Modifier'ları Gözden Geçirme

English: Make sure you know why everything in Table 5.4 is true. Use the first column for the first blank and the first row for the second blank. Also, remember that a member is a method or field.

Türkçe: Tablo 5.4'teki her sonucun neden doğru olduğunu bildiğinizden emin olun. İlk boşluk için ilk sütunu, ikinci boşluk için ilk satırı kullanın. Ayrıca bir member'ın method veya field olabileceğini unutmayın.

Source page 0243

English: TABLE 5.4 A method in _____ can access a _____ member.

Türkçe: TABLO 5.4 _____ içindeki bir method, _____ member'a erişebilir.

English: private package protected public

Türkçe: `private` · `package` · `protected` · `public`

English: the same class Yes Yes Yes Yes

Türkçe: aynı class · Evet · Evet · Evet · Evet

English: another class in the same package No Yes Yes Yes

Türkçe: aynı package'teki başka bir class · Hayır · Evet · Evet · Evet

English: a subclass in a different package No No Yes Yes

Türkçe: farklı package'teki bir subclass · Hayır · Hayır · Evet · Evet

English: an unrelated class in a different package No No No Yes

Türkçe: farklı package'teki ilişkisiz bir class · Hayır · Hayır · Hayır · Evet

Accessing static Data

Türkçe başlık: `static` Veriye Erişim

English: When the static keyword is applied to a variable, method, or class, it belongs to the class rather than a specific instance of the class. In this section, you see that the static keyword can also be applied to import statements.

Türkçe: `static` keyword'ü bir variable, method veya class'a uygulandığında üye belirli bir instance'a değil, class'a ait olur. Bu bölümde `static` keyword'ünün import statement'lara da uygulanabildiğini göreceksiniz.

Designing static Methods and Variables

Türkçe başlık: `static` Method ve Variable Tasarlama

English: Except for the `main()` method, we've been looking at instance methods. Methods and variables declared static don't require an instance of the class. They are shared among all users of the class. For instance, take a look at the following Penguin class:

Türkçe: `main()` dışında şimdiye kadar instance method'ları inceliyorduk. `static` bildirilen method ve variable'lar class instance'ı gerektirmez; class'ın bütün kullanıcıları arasında paylaşılır. Örneğin şu `Penguin` class'ına bakın:

```
public class Penguin {  
    String name;  
    static String nameOfTallestPenguin;  
}
```

English: In this class, every Penguin instance has its own name like Willy or Lilly, but only one Penguin among all the instances is the tallest. You can think of a static variable as being a member of the single class object that exists independently of any instances of that class. Consider the following example:

Türkçe: Bu class'ta her `Penguin` instance'ının Willy veya Lilly gibi kendine ait bir `name` value'su vardır; ancak bütün instance'lar içinde en uzun olan yalnızca bir `Penguin`'dir. `static` variable'ı, class'ın instance'larından bağımsız olarak var olan tek class object'inin bir member'ı gibi düşünebilirsiniz. Şu örneği inceleyin:

```
public static void main(String[] unused) {  
    var p1 = new Penguin();  
    p1.name = "Lilly";  
    p1.nameOfTallestPenguin = "Lilly";  
    var p2 = new Penguin();  
    p2.name = "Willy";  
    p2.nameOfTallestPenguin = "Willy";  
}
```

Source page 0244

```
System.out.println(p1.name); // Lilly  
System.out.println(p1.nameOfTallestPenguin); // Willy  
System.out.println(p2.name); // Willy  
System.out.println(p2.nameOfTallestPenguin); // Willy  
}
```

English: We see that each penguin instance is updated with its own unique name. The `nameOfTallestPenguin` field is static and therefore shared, though, so anytime it is updated, it impacts all instances of the class.

Türkçe: Her `Penguin` instance'ının kendine özgü `name` value'su vardır. Buna karşılık `nameOfTallestPenguin` field'ı `static`, dolayısıyla ortaktır; her güncelleme class'ın tüm instance'larını etkiler.

English: You have seen one static method since Chapter 1. The `main()` method is a static method. That means you can call it using the class name:

Türkçe: Chapter 1'den beri bir `static` method görüyorsunuz: `main().main()` bir `static` method olduğu için class name kullanılarak çağrılabilir:

```
public class Koala {  
    public static int count = 0; // static variable  
    public static void main(String[] args) { // static method  
        System.out.print(count);  
    }  
}
```

English: Here the JVM basically calls `Koala.main()` to get the program started. You can do this too. We can have a `KoalaTester` that does nothing but call the `main()` method:

Türkçe: JVM programı başlatmak için esasen `Koala.main()` çağrısı yapar. Siz de aynı çağrıyı yapabilirsiniz. Yalnızca `main()` method'unu çağırان bir `KoalaTester` yazabiliriz:

```
public class KoalaTester {  
    public static void main(String[] args) {  
        Koala.main(new String[0]); // call static method  
    }  
}
```

English: Quite a complicated way to print 0, isn't it? When we run `KoalaTester`, it makes a call to the `main()` method of `Koala`, which prints the value of `count`. The purpose of all these examples is to show that `main()` can be called just like any other static method.

Türkçe: 0 yazdırmak için epey karmaşık bir yol, değil mi? `KoalaTester` çalışınca `Koala` class'ının `main()` method'unu çağırır; bu method da `count` value'sunu yazdırır. Amaç, `main()` method'unun diğer `static` method'lar gibi çağrılabilirdiğini göstermektir.

English: In addition to `main()` methods, static methods have two main purposes:

Türkçe: `main()` dışında `static` method'ların iki temel kullanım amacı vardır:

English: • For utility or helper methods that don't require any object state. Since there is no need to access instance variables, having static methods eliminates the need for the caller to instantiate an object just to call the method.

Türkçe: • Object state gerektirmeyen utility/helper method'lar için. Instance variable'a erişme ihtiyacı yoksa `static` method kullanmak, caller'ın yalnızca method'u çağırarak amacıyla object instantiate etmesini gereksiz kılar.

English: • For state that is shared by all instances of a class, like a counter. All instances must share the same state. Methods that merely use that state should be static as well.

Türkçe: • Counter gibi class'ın bütün instance'larınca paylaşılan state için. Bütün instance'lar aynı state'i paylaşmalıdır; yalnızca bu state'i kullanan method'lar da `static` olmalıdır.

English: In the following sections, we look at some examples covering other static concepts.

Türkçe: Sonraki bölümlerde diğer `static` kavramları gösteren örnekleri inceleyeceğiz.

Accessing a static Variable or Method

Türkçe başlık: `static` Variable veya Method'a Erişim

English: Usually, accessing a static member is easy.

Türkçe: Bir `static` member'a erişmek genellikle kolaydır.

```
public class Snake {  
    public static long hiss = 2;  
}
```

Source page 0245

English: You just put the class name before the method or variable, and you are done. Here's an example:

Türkçe: Method veya variable'ın önüne class name'i yazmanız yeterlidir. Örneğin:

```
System.out.println(Snake.hiss);
```

English: Nice and easy. There is one rule that is trickier. You can use an instance of the object to call a static method. The compiler checks for the type of the reference and uses that instead of the object—which is sneaky of Java. This code is perfectly legal:

Türkçe: Buraya kadar kolay. Daha ince olan kural şudur: Bir `static` method veya variable'a object instance'ı üzerinden de erişilebilir. Compiler runtime object'e değil, reference type'a bakar ve class member'ını buna göre çözer. Bu nedenle aşağıdaki code tamamen geçerlidir:

```
5: Snake s = new Snake();  
6: System.out.println(s.hiss); // s is a Snake  
7: s = null;  
8: System.out.println(s.hiss); // s is still a Snake
```

English: Believe it or not, this code outputs 2 twice. Line 6 sees that `s` is a `Snake` and `hiss` is a static variable, so it reads that static variable. Line 8 does the same thing. Java doesn't care that `s` happens to be null. Since we are looking for a static variable, it doesn't matter.

Türkçe: İnanması güç olsa da kod iki kez 2 yazdırır. 6. satırda compiler, `s` reference type'ının `Snake`, `hiss`'in de `static` variable olduğunu görüp bu variable'ı okur. 8. satır da aynısını yapar. `s` value'sunun `null` olması önemli değildir; erişilen member `static` tir.

English: Remember to look at the reference type for a variable when you see a static method or variable. The exam creators will try to trick you into thinking a `NullPointerException` is thrown because the variable happens to be null. Don't be fooled!

Türkçe: `static` method veya variable gördüğünüzde variable'ın reference type'ına bakmayı unutmayın. Sınav, reference `null` olduğu için `NullPointerException` fırlatılacağını düşündürmeye çalışabilir; bu tuzağa düşmeyin.

English: One more time, because this is really important: what does the following output?

Türkçe: Bu kural çok önemli olduğu için bir kez daha soralım: Aşağıdaki code ne yazdırır?

```
Snake.hiss = 4;
Snake snake1 = new Snake();
Snake snake2 = new Snake();
snake1.hiss = 6;
snake2.hiss = 5;
System.out.println(Snake.hiss);
```

English: We hope you answered 5. There is only one hiss variable since it is static. It is set to 4 and then 6 and finally winds up as 5. All the Snake variables are just distractions.

Türkçe: Doğru cevap 5'tir. `hiss` `static` olduğundan yalnızca bir tane vardır; önce 4, sonra 6, en son 5 olur. Bütün `Snake` variable'ları dikkat dağıtıcıdır.

English: Class vs. Instance Membership

Türkçe: Class ve Instance Membership

English: There's another way the exam creators will try to trick you regarding static and instance members. A static member cannot call an instance member without referencing an instance of the class. This shouldn't be a surprise since static doesn't require any instances of the class to even exist.

Türkçe: Sınavda `static` ve instance member'lar konusunda başka bir tuzak daha vardır. Bir `static` member, class instance'ına başvurmadan instance member çağırabilir. Bu beklenen bir sonuçtur; çünkü `static` member'ın kullanılabilmesi için class'ın herhangi bir instance'ının var olması gerekmez.

English: The following is a common mistake for rookie programmers to make:

Türkçe: Aşağıdaki, yeni başlayan developer'ların sık yaptığı bir hatadır:

```
public class MantaRay {
    private String name = "Sammy";
    public static void first() { }
```

Source page 0246

```
public static void second() { }
public void third() { System.out.print(name); }
public static void main(String args[]) {
    first();
    second();
    third(); // DOES NOT COMPILE
}
```

English: The compiler will give you an error about making a static reference to an instance method. If we fix this by adding static to third(), we create a new problem. Can you figure out what it is?

Türkçe: Compiler, bir instance method'a `static` context'ten reference verildiğini belirten hata üretir. `third()` method'una `static` ekleyerek bunu düzeltirsek bu kez başka bir sorun doğar. Sorunu bulabiliyor musunuz?

```
public static void third() { System.out.print(name); } // DOES NOT COMPILE
```

English: All this does is move the problem. Now, `third()` is referring to an instance variable name. There are two ways we could fix this. The first is to add `static` to the name variable as well.

Türkçe: Bu yalnızca sorunu başka yere taşır. Artık `third()` instance variable `name`'e başvurmaktadır. İki çözüm vardır: ilki `name` variable'ına da `static` eklemektir.

```
public class MantaRay {
    private static String name = "Sammy";
    ...
    public static void third() { System.out.print(name); }
    ...
}
```

English: The second solution would have been to call `third()` as an instance method and not use `static` for the method or the variable.

Türkçe: İkinci çözüm, `third()` method'unu bir instance method olarak çağırmak ve ne method ne de variable için `static` kullanmaktır.

```
public class MantaRay {
    private String name = "Sammy";
    ...
    public void third() { System.out.print(name); }
    public static void main(String args[]) {
        ...
        var ray = new MantaRay();
        ray.third();
    }
}
```

English: The exam creators like this topic— a lot. A static method or instance method can call a static method because static methods don't require an object to use. Only an instance method can call another instance method on the same class without using a reference variable, because instance methods do require an object. Similar logic applies for instance and static variables.

Türkçe: Sınav bu konuyu çok sever. `static` method'lar object gerektirmediklerinden hem `static` hem instance method bir `static` method'u çağırabilir. Instance method'lar ise object gerektirir; bu yüzden aynı class'taki başka bir instance method'u explicit reference variable olmadan yalnızca bir instance method çağırabilir. Aynı mantık instance ve `static` variable'lar için de geçerlidir.

Source page 0247

English: Suppose we have a Giraffe class:

Türkçe: Bir `Giraffe` class'ımız olduğunu varsayalım:

```
public class Giraffe {
    public void eat(Giraffe g) {}
    public void drink() {};
    public static void allGiraffeGoHome(Giraffe g) {}
    public static void allGiraffeComeOut() {}
}
```

English: Make sure you understand Table 5.5 before continuing.
Türkçe: Devam etmeden önce Tablo 5.5'i anladığınızdan emin olun.

English: TABLE 5.5 Static vs. instance calls
Türkçe: TABLO 5.5 `static` ve instance call'lar

English: Method Calling Legal?
Türkçe: Çağırılan method · Çağırılan member · Geçerli mi?

English: `allGiraffeGoHome()` `allGiraffeComeOut()` Yes
Türkçe: `allGiraffeGoHome()` → `allGiraffeComeOut()` · Evet

English: `allGiraffeGoHome()` `drink()` No
Türkçe: `allGiraffeGoHome()` → `drink()` · Hayır

English: `allGiraffeGoHome()` `g.eat()` Yes
Türkçe: `allGiraffeGoHome()` → `g.eat()` · Evet

English: `eat()` `allGiraffeComeOut()` Yes
Türkçe: `eat()` → `allGiraffeComeOut()` · Evet

English: `eat()` `drink()` Yes
Türkçe: `eat()` → `drink()` · Evet

English: `eat()` `g.eat()` Yes
Türkçe: `eat()` → `g.eat()` · Evet

English: Let's try one more example so you have more practice at recognizing this scenario. Do you understand why the following lines fail to compile?
Türkçe: Bu senaryoyu tanıma konusunda biraz daha pratik yapmak için bir örnek daha deneyelim. Aşağıdaki satırların neden derlenmediğini anlıyor musunuz?

```

1: public class Gorilla {
2: public static int count;
3: public static void addGorilla() { count++; }
4: public void babyGorilla() { count++; }
5: public void announceBabies() {
6: addGorilla();
7: babyGorilla();
8: }
9: public static void announceBabiesToEveryone() {
10: addGorilla();
11: babyGorilla(); // DOES NOT COMPILE
12: }
13: public int total;
14: public static double average
15:     = total / count; // DOES NOT COMPILE
16: }

```

Source page 0248

English: Lines 3 and 4 are fine because both static and instance methods can refer to a static variable. Lines 5–8 are fine because an instance method can call a static method. Line 11 doesn't compile because a static method cannot call an instance method. Similarly, line 15 doesn't compile because a static variable is trying to use an instance variable.

Türkçe: 3. ve 4. satırlar geçerlidir; hem `static` hem de instance method bir `static` variable'a başvurabilir. 5–8. satırlar da geçerlidir; instance method bir `static` method'u çağırabilir. 11. satır derlenmez; çünkü `static` method, object reference olmadan instance method'u çağıramaz. Benzer biçimde 15. satır da derlenmez; `static` variable initializer, instance variable `total`'a erişmeye çalışır.

English: A common use for static variables is counting the number of instances:

Türkçe: `static` variable'ların yaygın bir kullanımı instance sayısını tutmaktır:

```

public class Counter {
private static int count;
public Counter() { count++; }
public static void main(String[] args) {
Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();
System.out.println(count); // 3
}
}

```

English: Each time the constructor is called, it increments count by one. This example relies on the fact that static (and instance) variables are automatically initialized to the default value for that type, which is 0 for int. See Chapter 1 to review the default values.

Türkçe: Constructor her çağrıldığında `count` bir artırılır. Örnek, `static` ve instance variable'ların kendi type'larının default value'suyla—`int` için 0—otomatik initialize edilmesine dayanır. Default value'lar için Chapter 1'i gözden geçirin.

English: Also notice that we didn't write `Counter.count`. We could have. It isn't necessary because we are already in that class, so the compiler can infer it.

Türkçe: Ayrıca `Counter.count` yazmadığımıza dikkat edin. Yazabilirdik; ancak zaten `Counter` class'ı içinde bulunduğumuzdan qualifier gerekli değildir ve compiler bunu çözer.

English: Make sure you understand this section really well. It comes up throughout this book. You even see a similar topic when we discuss interfaces in Chapter 7. For example, a static interface method cannot call a default interface method without a reference, much the same way that within a class, a static method cannot call an instance method without a reference.

Türkçe: Bu bölümü iyice anlayın; kural kitap boyunca yeniden karşınıza çıkacaktır. Chapter 7'deki interface'lerde de benzer bir durum vardır: Bir `static` interface method, object reference olmadan bir `default` interface method'u çağırabilir. Aynı şekilde class içindeki bir `static` method da object reference olmadan instance method'u çağırabilir.

static Variable Modifiers

Türkçe başlık: `static` Variable Modifier'ları

English: Referring back to Table 5.3, static variables can be declared with the same modifiers as instance variables, such as `final`, `transient`, and `volatile`. While some static variables are meant to change as the program runs, like our `count` example, others are meant to never change. This type of static variable is known as a constant. It uses the `final` modifier to ensure the variable never changes.

Türkçe: Tablo 5.3'e dönersek `static` variable'lar, instance variable'larla aynı `final`, `transient` ve `volatile` modifier'larıyla bildirilebilir. `count` örneğindeki gibi bazıları program çalışırken değişir; bazılarının ise hiç değişmemesi amaçlanır. Bu ikinci tür `static` variable constant olarak bilinir ve değişmemesi `final` modifier ile sağlanır.

English: Constants use the modifier `static final` and a different naming convention than other variables. They use all uppercase letters with underscores between "words." Here's an example:

Türkçe: Constant'lar `static final` modifier'larını ve diğer variable'lardan farklı bir naming convention kullanır: tüm harfler büyük, "kelimeler" arasında underscore olur. Örneğin:

Source page 0249

```
public class ZooPen {
    private static final int NUM_BUCKETS = 45;
    public static void main(String[] args) {
        NUM_BUCKETS = 5; // DOES NOT COMPILE
    }
}
```

English: The compiler will make sure that you do not accidentally try to update a final variable.

Türkçe: Compiler, bir `final` variable'ı yanlışlıkla güncellemeye çalışmanızı engeller.

English: This can get interesting. Do you think the following compiles?

Türkçe: Burada ilginç bir durum vardır. Sizce aşağıdaki code derlenir mi?

```
import java.util.*;
public class ZooInventoryManager {
    private static final String[] treats = new String[10];
    public static void main(String[] args) {
        treats[0] = "popcorn";
    }
}
```

English: It actually does compile since `treats` is a reference variable. We are allowed to modify the referenced object or array's contents. All the compiler can do is check that we don't try to reassign `treats` to point to a different object.

Türkçe: Kod gerçekten derlenir; çünkü `treats` bir reference variable'dır. İşaret edilen object'in veya array'in içeriği değiştirilebilir. Compiler yalnızca `treats` variable'ını başka bir object'e işaret edecek şekilde yeniden atamadığımızı denetler.

English: The rules for static final variables are similar to instance final variables, except they do not use static constructors (there is no such thing!) and use static initializers instead of instance initializers.

Türkçe: `static final` variable kuralları instance `final` variable kurallarına benzer. Fark, static constructor diye bir şey olmaması ve instance initializer yerine static initializer kullanılmasıdır.

```
public class Panda {
    final static String name = "Ronda";
    static final int bamboo;
    static final double height; // DOES NOT COMPILE
    static { bamboo = 5; }
}
```

English: The `name` variable is assigned a value when it is declared, while the `bamboo` variable is assigned a value in a static initializer. The `height` variable is not assigned a value anywhere in the class definition, so that line does not compile. Remember, final variables must be initialized with a value. Next, we cover static initializers.

Türkçe: `name` declaration sırasında, `bamboo` static initializer içinde değer alır. `height`, class definition'ın hiçbir yerinde değer almadığı için ilgili satır derlenmez. `final` variable'ların bir value ile initialize edilmesi gerektiğini unutmayın. Sırada static initializer'lar var.

Source page 0250

static Initializers

Türkçe başlık: `static` Initializer'lar

English: In Chapter 1, we covered instance initializers that looked like unnamed methods— just code inside braces. static initializers look similar. They add the static keyword to specify that they should be run when the class is first loaded. Here's an example:

Türkçe: Chapter 1'de, isimsiz method'lara benzeyen ve yalnızca brace'ler içinde code barındıran instance initializer'ları gördük. `static` initializer'lar da benzer görünür; class ilk yüklendiğinde çalışacaklarını belirtmek için `static` keyword'ü eklenir. Örneğin:

```
private static final int NUM_SECONDS_PER_MINUTE;
private static final int NUM_MINUTES_PER_HOUR;
private static final int NUM_SECONDS_PER_HOUR;
static {
    NUM_SECONDS_PER_MINUTE = 60;
    NUM_MINUTES_PER_HOUR = 60;
}
static {
    NUM_SECONDS_PER_HOUR
        = NUM_SECONDS_PER_MINUTE * NUM_MINUTES_PER_HOUR;
}
```

English: All static initializers run when the class is first used, in the order they are defined.

Türkçe: Bütün `static` initializer'lar class ilk kullanıldığında, source code'da bildirildikleri sırayla çalışır.

English: The statements in them run and assign any static variables as needed. There is something interesting about this example. We just got through saying that final variables aren't allowed to be reassigned. The key here is that the static initializer is the first assignment.

Türkçe: Bu block'lardaki statement'lar çalışır ve gerektiğinde `static` variable'lara value atar. Örnekteki önemli nokta şudur: `final` variable yeniden atanamaz; fakat burada `static` initializer variable'a ilk assignment'ı yapmaktadır.

English: And since it occurs up front, it is okay.

Türkçe: Bu assignment en başta yapıldığı için geçerlidir.

English: Let's try another example to make sure you understand the distinction:

Türkçe: Ayrımı anladığınızdan emin olmak için başka bir örnek deneyelim:

```
14: private static int one;
15: private static final int two;
16: private static final int three = 3;
17: private static final int four; // DOES NOT COMPILE
18: static {
19:     one = 1;
20:     two = 2;
21:     three = 3; // DOES NOT COMPILE
22:     two = 4;   // DOES NOT COMPILE
23: }
```

English: Line 14 declares a static variable that is not final. It can be assigned as many times as we like. Line 15 declares a final variable without initializing it. This means we can initialize it exactly once in a static block. Line 22 doesn't compile because this is the second attempt. Line 16 declares a final variable and initializes it at the same time. We are not allowed to assign it again, so line 21 doesn't compile. Line 17 declares a final variable that never gets initialized. The compiler gives a compiler error because it knows that the static blocks are the only place the variable could possibly be initialized. Since the programmer forgot, this is clearly an error.

Türkçe: 14. satır `final` olmayan bir `static` variable bildirir; bu variable istenildiği kadar yeniden atanabilir. 15. satır `final` variable `two`'yu initialize etmeden bildirir; dolayısıyla `static` block içinde tam bir kez initialize edilebilir. 22. satır ikinci assignment olduğu için derlenmez. 16. satır `final` variable `three`'yi declaration ile birlikte initialize eder; 21. satırdaki yeniden assignment bu nedenle derlenmez. 17. satırdaki `final` variable `four` ise hiç initialize edilmez. Compiler, bu variable'ın yalnızca `static` block'larda initialize edilebileceğini ve bunun hiç yapılmadığını bildiğinden compilation error verir.

Source page 0251

Try to Avoid static and Instance Initializers

Türkçe başlık: `static` ve Instance Initializer'lardan Kaçınmaya Çalışın

English: Using static and instance initializers can make your code much harder to read. Everything that could be done in an instance initializer could be done in a constructor instead.

Türkçe: `static` ve instance initializer kullanımı code'u okumayı oldukça zorlaştırabilir. Instance initializer içinde yapılabilen her şey constructor içinde de yapılabilir.

English: Many people find the constructor approach easier to read.

Türkçe: Birçok kişi constructor yaklaşımını daha okunabilir bulur.

English: There is a common case to use a static initializer: when you need to initialize a static field and the code to do so requires more than one line. This often occurs when you want to initialize a collection like an ArrayList or a HashMap. When you do need to use a static initializer, put all the static initialization in the same block. That way, the order is obvious.

Türkçe: `static` initializer'ın yaygın bir kullanım alanı, bir `static` field'ı initialize eden code'un birden fazla satır gerektirmesidir. Bu durum `ArrayList` veya `HashMap` gibi collection'lar initialize edilirken sık görülür. `static` initializer gerekiyorsa bütün `static` initialization code'unu aynı block'a koyun; böylece execution order açık olur.

static Imports

Türkçe başlık: `static` Import'lar

English: In Chapter 1, you saw that you can import a specific class or all the classes in a package.

Türkçe: Chapter 1'de belirli bir class'ı veya bir paketdeki tüm class'ları içe aktarabileceğinizi gördünüz.

English: If you haven't seen `ArrayList` or `List` before, don't worry, because we cover them in detail in Chapter 9, "Collections and Generics."

Türkçe: Daha önce `ArrayList` veya `List` görmediyseniz endişelenmeyin; bunları Chapter 9, "Collections and Generics" bölümünde ayrıntılı olarak ele alacağız.

```
import java.util.ArrayList;
import java.util.*;
```

English: We could use this technique to import two classes:

Türkçe: Bu tekniği iki class'ı içe aktarmak için kullanabiliriz:

```
import java.util.List;
import java.util.Arrays;
public class Imports {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("one", "two");
    }
}
```

English: Imports are convenient because you don't need to specify where each class comes from each time you use it. There is another type of import called a static import. Regular imports are for importing classes, while static imports are for importing static members of classes like variables and methods.

Türkçe: Import'lar kullanışlıdır; bir class'ı her kullandığınızda hangi package'ten geldiğini yazmanız gerekmez. Bir de static import vardır. Regular import class'ları, static import ise class'ların `static` variable ve method gibi `static` member'larını import eder.

English: Just like regular imports, you can use a wildcard or import a specific member. The idea is that you shouldn't have to specify where each static method or variable comes from each time you use it. An example of when static imports shine is when you are referring to a lot of constants in another class.

Türkçe: Regular import'ta olduğu gibi wildcard kullanabilir veya belirli bir member'ı import edebilirsiniz. Böylece her kullanımda `static` method ya da variable'ın geldiği class'ı belirtmek gerekmez. Static import, özellikle başka bir class'taki çok sayıda constant'a başvururken yararlıdır.

English: We can rewrite our previous example to use a static import. Doing so yields the following:

Türkçe: Önceki örneği static import kullanacak biçimde yeniden yazabiliriz. Sonuç şöyledir:

Editor note: Basılı kaynakta "We ran rewrite" yazan açık typo, anlamı koruyacak biçimde "We can rewrite" olarak düzeltilmiştir.

```
import java.util.List;
import static java.util.Arrays.asList; // static import
public class ZooParking {
    public static void main(String[] args) {
        List<String> list = asList("one", "two"); // No Arrays. prefix
    }
}
```

Source page 0252

English: In this example, we are specifically importing the `asList` method. This means that any time we refer to `asList` in the class, it will call `Arrays.asList()`.

Türkçe: Bu örnekte yalnızca `asList` method'unu import ediyoruz. Class içinde `asList`'e her başvuru `Arrays.asList()` method'unu çağırır.

English: An interesting case is what would happen if we created an `asList` method in our `ZooParking` class. Java would give it preference over the imported one, and the method we coded would be used.

Türkçe: `ZooParking` class'ında ayrıca kendi `asList` method'umuzu tanımlarsak Java local declaration'a import edilen method'dan önce öncelik verir; bizim yazdığımız method kullanılır.

English: The exam will try to trick you by misusing static imports. This example shows almost everything you can do wrong. Can you figure out what is wrong with each one?

Türkçe: Sınav static import'u yanlış kullanarak sizi yanıltmaya çalışabilir. Bu örnek neredeyse bütün olası hataları gösterir. Her satırdaki sorunu bulabiliyor musunuz?

```
1: import static java.util.Arrays; // DOES NOT COMPILE
2: import static java.util.Arrays.asList;
3: static import java.util.Arrays.*; // DOES NOT COMPILE
4: public class BadZooParking {
5:     public static void main(String[] args) {
6:         Arrays.asList("one"); // DOES NOT COMPILE
7:     }
8: }
```

English: Line 1 tries to use a static import to import a class. Remember that static imports are only for importing static members like a method or variable. Regular imports are for importing a class. Line 3 tries to see whether you are paying attention to the order of keywords. The syntax is `import static` and not vice versa. Line 6 is sneaky. The `asList` method is imported on line 2. However, the `Arrays` class is not imported anywhere. This makes it okay to write `asList("one")` but not `Arrays.asList("one")`.

Türkçe: 1. satır static import ile class import etmeye çalışır; oysa static import yalnızca method veya variable gibi static member'lar içindir, class için regular import kullanılır. 3. satır keyword sırasını ölçer: syntax `import static` tir, `static import` değildir. 6. satırdaki tuzak ise şudur: 2. satır `asList` method'unu import eder ama `Arrays` class'ını import etmez. Bu nedenle `asList("one")` yazılabilir, `Arrays.asList("one")` yazılamaz.

English: There's only one more scenario with static imports. In Chapter 1, you learned that importing two classes with the same name gives a compiler error. This is true of static imports as well. The compiler will complain if you try to explicitly do a static import of two methods with the same name or two static variables with the same name.

Türkçe: Static import hakkında bir senaryo daha vardır. Chapter 1'de aynı simple name'e sahip iki class'ı import etmenin compiler error verdiğini öğrendiniz. Aynı kural static import için de geçerlidir: Aynı name'e sahip iki method'u veya iki `static` variable'ı explicit static import ile getirmeye çalışırsanız compiler itiraz eder.

English: Here's an example:

Türkçe: İşte bir örnek:

```
import static zoo.A.TYPE;  
import static zoo.B.TYPE; // DOES NOT COMPILE
```

English: Luckily, when this happens, we can just refer to the static members via their class name in the code instead of trying to use a static import.

Türkçe: Böyle bir çakışmada static import kullanmak yerine `static` member'lara code içinde class name ile başvurabiliriz.

English: In a large program, static imports can be overused. When importing from too many places, it can be hard to remember where each static member comes from. Use them sparingly!

Türkçe: Büyük bir programda static import kolayca aşırı kullanılabilir. Çok fazla yerden import yapıldığında her `static` member'ın hangi class'tan geldiğini hatırlamak zorlaşır. Static import'u ölçülü kullanın.

Source page 0253

Passing Data among Methods

Türkçe başlık: Method'lar Arasında Data Aktarımı

English: Java is a “pass-by-value” language. This means that a copy of the variable is made and the method receives that copy. Assignments made in the method do not affect the caller. Let's look at an example:

Türkçe: Java pass-by-value (değere göre aktarım) kullanır. Variable value'sunun bir kopyası oluşturulur ve method bu kopyayı alır. Method içindeki atamalar caller'ı etkilemez. Bir örneğe bakalım:

```
2: public static void main(String[] args) {  
3:   int num = 4;  
4:   newNumber(num);  
5:   System.out.print(num); // 4  
6: }  
7: public static void newNumber(int num) {  
8:   num = 8;  
9: }
```

English: On line 3, num is assigned the value of 4. On line 4, we call a method. On line 8, the num parameter in the method is set to 8. Although this parameter has the same name as the variable on line 3, this is a coincidence. The name could be anything. The exam will often use the same name to try to confuse you. The variable on line 3 never changes because no assignments are made to it.

Türkçe: 3. satırda `num` value'su 4 olur. 4. satır bir method çağırır; 8. satırda method'un `num` parameter'ına 8 atanır. Bu parameter'ın 3. satırdaki variable ile aynı adı taşıması yalnızca bir tesadüftür; adı farklı da olabilirdi. Sınav kafa karıştırmak için sıkça aynı adı kullanır. 3. satırdaki variable'a hiçbir assignment yapılmadığından value'su değişmez.

Passing Objects

Türkçe başlık: Object'leri Geçirme

English: Now that you've seen primitives, let's try an example with a reference type. What do you think is output by the following code?

Türkçe: Primitive'leri gördüğünüze göre şimdi bir reference type örneği deneyelim. Aşağıdaki kod sizce ne yazdırır?

```
public class Dog {
    public static void main(String[] args) {
        String name = "Webby";
        speak(name);
        System.out.print(name);
    }
    public static void speak(String name) {
        name = "Georgette";
    }
}
```

English: The correct answer is Webby. Just as in the primitive example, the variable assignment is only to the method parameter and doesn't affect the caller.

Türkçe: Doğru cevap **webby**'dir. Primitive örnekte olduğu gibi assignment yalnızca method parameter'ına yapılır; caller'daki variable etkilenmez.

Source page 0254

English: Notice how we keep talking about variable assignments. This is because we can call methods on the parameters. As an example, here is code that calls a method on the `StringBuilder` passed into the method:

Türkçe: Sürekli variable assignment'lardan söz ettiğimize dikkat edin. Çünkü parameter object'i üzerinde method çağırmak da mümkündür. Aşağıdaki kod, method'a geçirilen `StringBuilder` üzerinde bir method çağırır:

```
public class Dog {
    public static void main(String[] args) {
        var name = new StringBuilder("Webby");
        speak(name);
        System.out.print(name); // WebbyGeorgette
    }
    public static void speak(StringBuilder s) {
        s.append("Georgette");
    }
}
```

English: In this case, `speak()` calls a method on the parameter. It doesn't reassign `s` to a different object. In Figure 5.4, you can see how pass-by-value is still used. The variable `s` is a copy of the variable name. Both point to the same `StringBuilder`, which means that changes made to the `StringBuilder` are available to both references.

Türkçe: Bu durumda `speak()`, parameter üzerinde method çağırır; `s` reference'ını başka bir object'e yeniden atamaz. Şekil 5.4, pass-by-value'ın hâlâ geçerli olduğunu gösterir. `s`, `name` variable'ındaki reference value'nun bir kopyasını taşır. İki reference da aynı `StringBuilder` object'ini gösterdiğinden object'teki değişiklikler iki reference üzerinden de görülür.

English: FIGURE 5.4 Copying a reference with pass-by-value

Türkçe: ŞEKİL 5.4 Pass-by-value ile bir reference'ın kopyalanması

English: name

Türkçe: name

English: `StringBuilder`

Türkçe: `StringBuilder`

English: object

Türkçe: object

English: `s`

Türkçe: `s`

English: Pass-by-Value vs. Pass-by-Reference

Türkçe: Pass-by-Value ve Pass-by-Reference

English: Different languages handle parameters in different ways. Pass-by-value is used by many languages, including Java. In this example, the `swap()` method does not change the original values. It only changes `a` and `b` within the method.

Türkçe: Farklı diller parameter'ları farklı biçimlerde ele alır. Java dahil birçok dil pass-by-value kullanır. Bu örnekte `swap()` caller'daki asıl value'ları değiştirmez; yalnızca method içindeki `a` ve `b` parameter'larını değiştirir.

```
public static void main(String[] args) {
    int original1 = 1;
    int original2 = 2;
    swap(original1, original2);
    System.out.println(original1); // 1
    System.out.println(original2); // 2
}

public static void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}
```

Source page 0255

English: The other approach is pass-by-reference. It is used by default in a few languages, such as Perl. We aren't going to show you Perl code here because you are studying for the Java exam, and we don't want to confuse you. In a pass-by-reference language, the variables would be swapped and the output would be reversed.

Türkçe: Diğer yaklaşım pass-by-reference'tır. Perl gibi bazı diller bunu varsayılan olarak kullanır. Java sınavına hazırlandığınız için burada Perl kodu göstermeyeceğiz. Pass-by-reference kullanan bir dilde variable'lar yer değiştirir ve çıktı ters sırada olurdu.

English: To review, Java uses pass-by-value to get data into a method. Assigning a new primitive or reference to a parameter doesn't change the caller. Calling methods on a reference to an object can affect the caller.

Türkçe: Özetle Java, veriyi method'a aktarmak için pass-by-value kullanır. Parameter'a yeni bir primitive value veya reference atamak caller'ı değiştirmez. Ancak bir object reference'ı üzerinden method çağırmak object state'ini değiştirerek caller tarafında görülebilir.

Returning Objects

Türkçe başlık: Object Döndürme

English: Getting data back from a method is easier. A copy is made of the primitive or reference and returned from the method. Most of the time, this returned value is used. For example, it might be stored in a variable. If the returned value is not used, the result is ignored. Watch for this on the exam. Ignored returned values are tricky.

Türkçe: Method'dan veri geri almak daha kolaydır: primitive value'nun veya reference'ın bir kopyası method'dan döndürülür. Çoğu zaman bu return value kullanılır; örneğin bir variable'da saklanır. Return value kullanılmazsa sonuç yok sayılır. Sınavda buna dikkat edin; göz ardı edilen return value'lar yanıltıcı olabilir.

English: Let's try an example. Pay attention to the return types.

Türkçe: Bir örnek deneyelim. Return type'lara dikkat edin.

```
1: public class ZooTickets {
2:     public static void main(String[] args) {
3:         int tickets = 2; // tickets = 2
4:         String guests = "abc"; // guests = abc
5:         addTickets(tickets); // tickets = 2
6:         guests = addGuests(guests); // guests = abcd
7:         System.out.println(tickets + guests); // 2abcd
8:     }
9:     public static int addTickets(int tickets) {
10:         tickets++;
11:         return tickets;
12:     }
13:     public static String addGuests(String guests) {
14:         guests += "d";
15:         return guests;
16:     }
17: }
```

English: This is a tricky one because there is a lot to keep track of. When you see such questions on the exam, write down the values of each variable. Lines 3 and 4 are straightforward assignments. Line 5 calls a method. Line 10 increments the method parameter to 3 but leaves the tickets variable in the main() method as 2. While line 11 returns the value, the caller ignores it. The method call on line 6 doesn't ignore the result, so guests becomes "abcd". Remember that this is happening because of the returned value and not the method parameter.

Türkçe: Bu örnekte izlenecek birçok value vardır; sınavda benzer bir soru görürseniz her variable'ın value'sunu not edin. 3. ve 4. satırlar normal assignment'lardır. 5. satır `addTickets()` method'unu çağırır. 10. satır method parameter'ını 3 yapar, ancak `main()` içindeki `tickets` variable'ı 2 kalır. 11. satır bu value'yu döndürse de caller return value'yu yok sayar. 6. satır ise method sonucunu `guests` variable'ına atar; bu nedenle `guests` value'su "abcd" olur. Değişikliğin parameter assignment'ından değil, kullanılan return value'dan kaynaklandığını unutmayın.

Source page 0256

Autoboxing and Unboxing Variables

Türkçe başlık: Variable'larda Autoboxing ve Unboxing

English: Java supports some helpful features around passing primitive and wrapper data types, such as `int` and `Integer`. Remember from Chapter 1 that we can explicitly convert between primitives and wrapper classes using built-in methods.

Türkçe: Java, `int` ve `Integer` gibi primitive ve wrapper data type'ların aktarımını kolaylaştıran özellikler sunar. Chapter 1'den, built-in method'larla primitive'ler ve wrapper class'lar arasında açıkça conversion yapabildiğimizi hatırlayın.

```
5: int quack = 5;
6: Integer quackquack = Integer.valueOf(quack); // Convert int to Integer
7: int quackquackquack = quackquack.intValue(); // Convert Integer to int
```

English: Useful, but a bit verbose. Luckily, Java has handlers built into the Java language that automatically convert between primitives and wrapper classes and back again. Autoboxing is the process of converting a primitive into its equivalent wrapper class, while unboxing is the process of converting a wrapper class into its equivalent primitive.

Türkçe: Faydalı, fakat biraz uzun. Neyse ki Java primitive ile wrapper class arasında iki yönde de otomatik conversion yapar. Autoboxing, primitive'i karşılık gelen wrapper class'a; unboxing ise wrapper class'ı karşılık gelen primitive'e dönüştürme işlemidir.

```
5: int quack = 5;
6: Integer quackquack = quack; // Autoboxing
7: int quackquackquack = quackquack; // Unboxing
```

English: The new code is equivalent to the previous code, as the compiler is "doing the work" of converting the types automatically for you. Autoboxing applies to all primitives and their associated wrapper types, such as the following:

Türkçe: Compiler type conversion işini otomatik yaptığı için yeni code önceki code'a eşdeğerdir. Autoboxing bütün primitive type'lar ve bunlara karşılık gelen wrapper type'lar için geçerlidir. Örneğin:

```
Short tail = 8; // Autoboxing
Character p = Character.valueOf('p');
char paw = p; // Unboxing
Boolean nose = true; // Autoboxing
Integer e = Integer.valueOf(9);
long ears = e; // Unboxing, then implicit casting
```

English: Each of these examples compiles without issue. In the last line, `e` is unboxed to an `int` value.

Since an `int` value can be stored in a `long` variable via implicit casting, the compiler allows the assignment.

Türkçe: Bu örneklerin hepsi sorunsuz derlenir. Son satırda `e`, `int` value'ya unbox edilir. `int` value implicit cast ile `long` variable'da saklanabildiği için compiler assignment'a izin verir.

Limits of Autoboxing and Numeric Promotion

Türkçe başlık: Autoboxing ve Numeric Promotion Sınırları

English: While Java will implicitly cast a smaller primitive to a larger type, as well as autobox, it will not do both at the same time. Do you see why the following does not compile?

Türkçe: Java daha dar bir primitive type'ı daha geniş bir primitive type'a implicit cast edebilir ve ayrıca autoboxing yapabilir; fakat bu iki conversion'ı aynı anda yapmaz. Aşağıdaki declaration'ın neden derlenmediğini görüyor musunuz?

```
Long badGorilla = 8; // DOES NOT COMPILE
```

English: Java will automatically cast or autobox the `int` value to `long` or `Integer`, respectively.

Türkçe: Java `int` value'yu sırasıyla `long`a implicit cast edebilir veya `Integer`a autobox edebilir.

English: Neither of these types can be assigned to a `Long` reference variable, though, so the code does not compile. Compare this behavior to the previous example with `ears`, where the unboxed primitive value could be implicitly cast to a larger primitive type.

Türkçe: Ancak bu iki type'tan hiçbir `Long` reference variable'a atanamaz; bu nedenle code derlenmez. Bunu önceki `ears` örneğiyle karşılaştırın: Orada unbox edilen primitive value daha geniş bir primitive type'a implicit cast edilebiliyordu.

Source page 0257

English: What do you think happens if you try to unbox a `null`?

Türkçe: `null` bir reference'ı unbox etmeye çalışırsanız sizce ne olur?

```
10: Character elephant = null;
11: char badElephant = elephant; // NullPointerException
```

English: On line 10, we store null in a Character reference. This is legal because a null reference can be assigned to any reference variable. On line 11, we try to unbox that null to a char primitive. This is a problem. Java tries to get the char value of null. Since calling any method on null gives a NullPointerException, that is just what we get. Be careful when you see null in relation to autoboxing and unboxing.

Türkçe: 10. satırda `Character` reference'ına `null` atamak yasaldır; çünkü her reference variable `null` alabilir. 11. satırda bu `null` value `char` primitive'e unbox edilmeye çalışılır. Java `null`un `char` value'sunu almaya çalıştığı için `NullPointerException` fırlatılır. Autoboxing/unboxing ile birlikte `null` gördüğünüzde dikkatli olun.

English: Where autoboxing and unboxing really shine is when we apply them to method calls.

Türkçe: Autoboxing ve unboxing özellikle method call'larda kullanışlıdır.

```
public class Chimpanzee {
    public void climb(long t) {}
    public void swing(Integer u) {}
    public void jump(int v) {}
    public static void main(String[] args) {
        var c = new Chimpanzee();
        c.climb(123);
        c.swing(123);
        c.jump(123L); // DOES NOT COMPILE
    }
}
```

English: In this example, the call to `climb()` compiles because the `int` value can be implicitly cast to a `long`. The call to `swing()` also is permitted, because the `int` value is autoboxed to an `Integer`.

Türkçe: Bu örnekte `climb()` çağrısı derlenir; çünkü `int` value implicit olarak `long`a cast edilebilir. `swing()` çağrısına da izin verilir; çünkü `int` value `Integer`a autobox edilir.

English: On the other hand, the call to `jump()` results in a compiler error because a `long` must be explicitly cast to an `int`. In other words, Java will not automatically convert to a narrower type.

Türkçe: Buna karşılık `jump()` call'u compilation error verir; çünkü `long` value'nun `int`e explicit cast edilmesi gerekir. Java otomatik narrowing conversion yapmaz.

English: As before, the same limitation around autoboxing and numeric promotion applies to method calls. For example, the following does not compile:

Türkçe: Autoboxing ile numeric promotion'ı aynı anda yapamama sınırı method call'lar için de geçerlidir. Örneğin aşağıdaki code derlenmez:

```
public class Gorilla {
    public void rest(Long x) {
        System.out.print("long");
    }
    public static void main(String[] args) {
        var g = new Gorilla();
        g.rest(8); // DOES NOT COMPILE
    }
}
```

English: Java will cast or autobox the value automatically, but not both at the same time.

Türkçe: Java value'yu otomatik olarak cast veya autobox eder; ikisini aynı çağrıda birlikte yapmaz.

Source page 0258

Overloading Methods

Türkçe başlık: Method Overloading

English: Now that you are familiar with the rules for declaring and using methods, it is time to look at creating methods with the same name in the same class. Method overloading occurs when methods in the same class have the same name but different method signatures, which means they use different parameter lists. (Overloading differs from overriding, which you learn about in Chapter 6.)

Türkçe: Method bildirme ve kullanma kurallarını öğrendiğimize göre aynı class içinde aynı adlı method'lar oluşturmaya geçebiliriz. Method overloading, aynı class'taki method'ların aynı ada fakat farklı method signature'lara—yani farklı parameter list'lere—sahip olmasıdır. Overloading, Chapter 6'da göreceğiniz overriding'den farklıdır.

English: We've been showing how to call overloaded methods for a while. `System.out.println()` and `StringBuilder's append()` methods provide many overloaded versions, so you can pass just about anything to them without having to think about it. In both of these examples, the only change was the type of the parameter. Overloading also allows different numbers of parameters.

Türkçe: Bir süredir overloaded method çağrılarını zaten kullanıyoruz. `System.out.println()` ve `StringBuilder.append()` pek çok overloaded sürüm sunar; bu nedenle neredeyse her tür argument'i ayrıca düşünmeden geçebilirsiniz. Bu iki örnekte değişen yalnızca parameter type'tır. Overloading, parameter sayısının farklı olmasına da izin verir.

English: Everything other than the method name can vary for overloading methods. This means there can be different access modifiers, optional specifiers (like `static`), return types, and exception lists.

Türkçe: Overload'larda method name dışındaki her şey değişebilir: access modifier, `static` gibi optional specifier, return type ve exception list farklı olabilir.

English: The following shows five overloaded versions of the `fly()` method:

Türkçe: Aşağıda `fly()` method'unun beş overloaded sürümü gösterilir:

```
public class Falcon {
    public void fly(int numMiles) {}
    public void fly(short numFeet) {}
    public boolean fly() { return false; }
    void fly(int numMiles, short numFeet) {}
    public void fly(short numFeet, int numMiles) throws Exception {}
}
```

English: As you can see, we can overload by changing anything in the parameter list. We can have a different type, more types, or the same types in a different order. Also notice that the return type, access modifier, and exception list are irrelevant to overloading. Only the method name and parameter list matter.

Türkçe: Parameter list içindeki herhangi bir şeyi değiştirerek overload oluşturabiliriz: type farklı olabilir, type sayısı artabilir veya aynı type'ların sırası değişebilir. Return type, access modifier ve exception list overloading açısından önemsizdir; yalnızca method name ve parameter list belirleyicidir.

English: Now let's look at an example that is not valid overloading:

Türkçe: Şimdi geçerli bir overload oluşturmayan örneğe bakalım:

```
public class Eagle {  
    public void fly(int numMiles) {}  
    public int fly(int numMiles) { return 1; } // DOES NOT COMPILE  
}
```

English: This method doesn't compile because it differs from the original only by return type. The method signatures are the same, so they are duplicate methods as far as Java is concerned.

Türkçe: Bu method yalnızca return type bakımından ilk declaration'dan farklı olduğu için derlenmez. Method signature'ları aynıdır; dolayısıyla Java bunları duplicate method declaration olarak görür.

English: What about these; why do they not compile?

Türkçe: Peki ya bunlar? Neden derlenmezler?

```
public class Hawk {  
    public void fly(int numMiles) {}  
    public static void fly(int numMiles) {} // DOES NOT COMPILE  
    public void fly(int numKilometers) {} // DOES NOT COMPILE  
}
```

Source page 0259

English: Again, the method signatures of these three methods are the same. You cannot declare methods in the same class where the only difference is that one is an instance method and one is a static method. You also cannot have two methods that have parameter lists with the same variable types and in the same order. As we mentioned earlier, the names of the parameters in the list do not matter when determining the method signature.

Türkçe: Yine üç method'un da method signature'ı aynıdır. Aynı class içinde yalnızca biri instance, diğeri `static` olduğu için farklılaşan method'lar bildirilemez. Parameter type'ları ve sıraları aynı olan iki method da bildirilemez. Daha önce belirttiğimiz gibi parameter name'ler method signature'ı belirlerken dikkate alınmaz.

English: Calling overloaded methods is easy. You just write code, and Java calls the right one.

Türkçe: Overloaded method'ları çağırmak kolaydır. Code'u yazarsınız, Java uygun overload'ı seçer.

English: For example, look at these two methods:

Türkçe: Örneğin şu iki method'a bakın:

```
public class Dove {  
    public void fly(int numMiles) {  
        System.out.println("int");  
    }  
    public void fly(short numFeet) {  
        System.out.println("short");  
    }  
}
```

English: The call `fly((short) 1)` prints `short`. It looks for matching types and calls the appropriate method. Of course, it can be more complicated than this.

Türkçe: `fly((short) 1)` çağırısı `short` yazdırır. Java eşleşen type'ı bulup uygun method'u çağırır. Elbette seçim bundan daha karmaşık olabilir.

English: Now that you know the basics of overloading, let's look at some more complex scenarios that you may encounter on the exam.

Türkçe: Overloading'in temellerini bildiğinize göre sınavda karşılaşılabileceğiniz daha karmaşık senaryolara bakalım.

Reference Types

Türkçe başlık: Reference Type'lar

English: Given the rule about Java picking the most specific version of a method that it can, what do you think this code outputs?

Türkçe: Java'nın uygulanabilir en specific method sürümünü seçtiği kuralına göre sizce bu code ne yazdırır?

```
public class Pelican {
    public void fly(String s) {
        System.out.print("string");
    }
    public void fly(Object o) {
        System.out.print("object");
    }
    public static void main(String[] args) {
        var p = new Pelican();
        p.fly("test");
        System.out.print("-");
        p.fly(56);
    }
}
```

Source page 0260

English: The answer is `string-object`. The first call passes a `String` and finds a direct match.

Türkçe: Cevap `string-object`tir. İlk call bir `String` argument geçirir ve exact match bulur.

English: There's no reason to use the Object version when there is a nice String parameter list just waiting to be called. The second call looks for an int parameter list. When it doesn't find one, it autoboxes to Integer. Since it still doesn't find a match, it goes to the Object one.

Türkçe: Doğrudan eşleşen `String` parameter list varken `Object` sürümünü seçmek için neden yoktur. İkinci çağrı önce `int` parameter list arar; bulamayınca value `Integer`a autobox edilir. Yine eşleşme bulunamayınca `Object` overload'ı seçilir.

English: Let's try another. What does this print?

Türkçe: Başka bir örnek deneyelim. Bu code ne yazdırır?

```
import java.time.*;
import java.util.*;
public class Parrot {
    public static void print(List<Integer> i) {
        System.out.print("I");
    }
    public static void print(CharSequence c) {
        System.out.print("C");
    }
    public static void print(Object o) {
        System.out.print("O");
    }
    public static void main(String[] args){
        print("abc");
        print(Arrays.asList(3));
        print(LocalDate.of(2019, Month.JULY, 4));
    }
}
```

English: The answer is CIO. The code is due for a promotion! The first call to `print()` passes a `String`. As you learned in Chapter 4, `String` and `StringBuilder` implement the `CharSequence` interface. You also learned that `Arrays.asList()` can be used to create a `List<Integer>` object, which explains the second output. The final call to `print()` passes a `LocalDate`. This is a class you might not know, but that's okay. It clearly isn't a sequence of characters or a list. That means the `Object` method signature is used.

Türkçe: Cevap CIOdur; code gerçekten bir "promotion" hak ediyor! İlk `print()` call'u bir `String` geçirir. Chapter 4'ten bildiğiniz gibi `String` ve `StringBuilder`, `CharSequence` interface'ini implement eder. `Arrays.asList()` ise `List<Integer>` object'i üretir; ikinci output bu nedenle I olur. Son call bir `LocalDate` geçirir. Bu object ne `CharSequence` ne de `List` olduğundan `Object` method signature'ı seçilir.

English: Primitives

Türkçe: Primitive'ler

English: Primitives work in a way that's similar to reference variables. Java tries to find the most specific matching overloaded method. What do you think happens here?

Türkçe: Primitive'ler reference variable'lara benzer biçimde ele alınır. Java en specific matching overload'ı bulmaya çalışır. Sizce burada ne olur?

```
public class Ostrich {
    public void fly(int i) {
        System.out.print("int");
    }
    public void fly(long l) {
        System.out.print("long");
    }
}
```

Source page 0261

```
public static void main(String[] args) {
    var p = new Ostrich();
    p.fly(123);
    System.out.print("-");
    p.fly(123L);
}
```

English: The answer is `int-long`. The first call passes an `int` and sees an exact match. The second call passes a `long` and also sees an exact match. If we comment out the overloaded method with the `int` parameter list, the output becomes `long-long`. Java has no problem calling a larger primitive. However, it will not do so unless a better match is not found.

Türkçe: Cevap `int-long`dur. İlk call `int`, ikinci call `long` argument geçirir ve ikisi de exact match bulur. `int` parameter list'li overload comment'e alınırsa output `long-long` olur. Java daha geniş bir primitive overload'ı çağırabilir; fakat bunu yalnızca daha iyi bir match yoksa yapar.

English: Autoboxing

Türkçe: Autoboxing

English: As we saw earlier, autoboxing applies to method calls, but what happens if you have both a primitive and an integer version?

Türkçe: Daha önce gördüğümüz gibi autoboxing method call'lara uygulanır. Peki hem primitive hem de `Integer` sürümü varsa ne olur?

```
public class Kiwi {
    public void fly(int numMiles) {}
    public void fly(Integer numMiles) {}
}
```

English: These method overloads are valid. Java tries to use the most specific parameter list it can find. This is true for autoboxing as well as other matching types we talk about in this section.

Türkçe: Bu method overload'ları geçerlidir. Java bulabildiği en specific parameter list'i seçer. Bu kural autoboxing ve bu bölümdeki diğer eşleşme türleri için de geçerlidir.

English: This means calling `fly(3)` will call the first method. When the primitive `int` version isn't present, Java will autobox. However, when the primitive `int` version is provided, there is no reason for Java to do the extra work of autoboxing.

Türkçe: Bu nedenle `fly(3)` ilk method'u çağırır. Primitive `int` sürümü yoksa Java autoboxing yapar; ancak exact primitive sürüm varken ek autoboxing işlemine gerek yoktur.

English: Arrays

Türkçe: Array'ler

English: Unlike the previous example, this code does not autobox:

Türkçe: Önceki örnekten farklı olarak bu kod autoboxing yapmaz:

```
public static void walk(int[] ints) {}  
public static void walk(Integer[] integers) {}
```

English: Arrays have been around since the beginning of Java. They specify their actual types.

Türkçe: Array'ler Java'nın ilk sürümlerinden beri vardır ve gerçek component type'larını belirtir.

English: What about generic types, such as `List<Integer>`? We cover this topic in Chapter 9.

Türkçe: `List<Integer>` gibi generic type'lara ne olur? Bu konuyu Chapter 9'da ele alacağız.

English: Varargs

Türkçe: Varargs

English: Which method do you think is called if we pass an `int[]`?

Türkçe: Bir `int[]` argument geçirirsek sizce hangi method çağrılır?

```
public class Toucan {  
    public void fly(int[] lengths) {}  
    public void fly(int... lengths) {} // DOES NOT COMPILE  
}
```

Source page 0262

English: Trick question! Remember that Java treats varargs as if they were an array. This means the method signature is the same for both methods. Since we are not allowed to overload methods with the same parameter list, this code doesn't compile. Even though the code doesn't look the same, it compiles to the same parameter list.

Türkçe: Bu bir tuzak sorudur. Java varargs'ı array gibi ele alır; dolayısıyla iki method'un signature'ı aynıdır. Aynı parameter list'e sahip method'lar overload edilemediği için kod derlenmez. Source code farklı görünse de compiler açısından parameter list aynıdır.

English: Now that we've just gotten through explaining that the two methods are similar, it is time to mention how they are different. It shouldn't be a surprise that you can call either method by passing an array:

Türkçe: İki method'un benzer olduğunu açıkladığımıza göre şimdi aralarındaki farktan söz edebiliriz. Bir array geçirerek iki method'dan herhangi birini çağırabilmeniz şaşırtıcı olmamalıdır:

```
fly(new int[] { 1, 2, 3 }); // Allowed to call either fly() method
```

English: However, you can only call the varargs version with stand-alone parameters:

Türkçe: Buna karşılık stand-alone parameter'larla yalnızca varargs sürümünü çağırabilirsiniz:

```
fly(1, 2, 3); // Allowed to call only the fly() method using varargs
```

English: Obviously, this means they don't compile exactly the same. The parameter list is the same, though, and that is what you need to know with respect to overloading for the exam.

Türkçe: Açıkça görüldüğü gibi ikisi tam olarak aynı şekilde derlenmez. Yine de parameter list aynıdır ve sınavda overloading bakımından bilmeniz gereken nokta budur.

Putting It All Together

Türkçe başlık: Hepsini Bir Araya Getirmek

English: So far, all the rules for when an overloaded method is called should be logical. Java calls the most specific method it can. When some of the types interact, the Java rules focus on backward compatibility. A long time ago, autoboxing and varargs didn't exist. Since old code still needs to work, this means autoboxing and varargs come last when Java looks at overloaded methods. Ready for the official order? Table 5.6 lays it out for you.

Türkçe: Buraya kadarki overload seçimi mantıklıdır: Java çağırabildiği en specific method'u seçer. Type'lar etkileştiğinde backward compatibility önem kazanır. Autoboxing ve varargs eski Java sürümlerinde yoktu; eski kodun çalışmaya devam etmesi gerektiğinden overload aramasında bunlar en sona bırakılır. Resmî sıra Tablo 5.6'dadır.

English: TABLE 5.6 The order that Java uses to choose the right overloaded method

Türkçe: TABLO 5.6 Java'nın doğru overloaded method'u seçme sırası

English: Rule · Example of what will be chosen for `glide(1,2)`

Türkçe: Kural · `glide(1,2)` için seçilecek declaration örneği

English: Exact match by type · `String glide(int i, int j)`

Türkçe: Type'a göre exact match · `String glide(int i, int j)`

English: Larger primitive type · `String glide(long i, long j)`

Türkçe: Daha geniş primitive type · `String glide(long i, long j)`

English: Autoboxed type · `String glide(Integer i, Integer j)`

Türkçe: Autoboxed type · `String glide(Integer i, Integer j)`

English: Varargs · `String glide(int... nums)`

Türkçe: Varargs · `String glide(int... nums)`

English: Let's give this a practice run using the rules in Table 5.6. What do you think this outputs?

Türkçe: Tablo 5.6'daki kurallarla bir alıştırma yapalım. Sizce bu code ne yazdırır?

```
public class Glider {  
    public static String glide(String s) {  
        return "1";  
    }  
    public static String glide(String... s) {  
        return "2";  
    }  
}
```

Source page 0263

```
public static String glide(Object o) {  
    return "3";  
}  
public static String glide(String s, String t) {  
    return "4";  
}  
public static void main(String[] args) {  
    System.out.print(glide("a"));  
    System.out.print(glide("a", "b"));  
    System.out.print(glide("a", "b", "c"));  
}  
}
```

English: It prints out 142. The first call matches the signature taking a single String because that is the most specific match. The second call matches the signature taking two String parameters since that is an exact match. It isn't until the third call that the varargs version is used since there are no better matches.

Türkçe: Code 142 yazdırır. İlk call, en specific match olan tek String parameter'lı signature'ı seçer. İkinci call iki String parameter'lı signature ile exact match olur. Daha iyi bir match bulunmadığı için varargs sürümü ancak üçüncü call'da kullanılır.

Summary

Türkçe başlık: Özet

English: In this chapter, we presented a lot of rules for declaring methods and variables. Methods start with access modifiers and optional specifiers in any order (although commonly with access modifiers first). The access modifiers we discussed in this chapter are `private`, `package` (omitted), `protected`, and `public`. The optional specifier for methods we covered in this chapter is `static`. We cover additional method modifiers in future chapters.

Türkçe: Bu chapter'da method ve variable declaration için birçok kural sunduk. Method'lar access modifier ve optional specifier'larla başlar; bunlar herhangi bir sırada yazılabilse de genellikle access modifier önce gelir. Ele aldığımız access düzeyleri `private`, `package` (modifier yazılmaz), `protected` ve `public`; optional method specifier ise `static`'tir. Diğer method modifier'larını sonraki chapter'larda göreceğiz.

English: Next comes the method return type, which is `void` if there is no return value. The method name and parameter list are provided next, which compose the unique method signature.

Türkçe: Ardından method return type gelir; return value yoksa bu type `void` olur. Sonra benzersiz method signature'ı oluşturan method name ve parameter list yazılır.

English: The method name uses standard Java identifier rules, while the parameter list is composed of zero or more types with names. An optional list of exceptions may also be added following the parameter list. Finally, a block defines the method body (which is omitted for abstract methods).

Türkçe: Method name standart Java identifier kurallarına uyar. Parameter list, her biri type ve name içeren sıfır veya daha fazla parameter'dan oluşur. Parameter list'ten sonra optional exception list gelebilir. Son olarak bir block method body'yi tanımlar; `abstract` method'larda body bulunmaz.

English: Access modifiers are used for a lot more than just methods, so make sure you understand them well. Using the `private` keyword means the code is only available from within the same class. `Package` access means the code is available only from within the same package.

Türkçe: Access modifier'lar yalnızca method'larda kullanılmaz; bu nedenle onları iyi anlayın. `private` kodu aynı class ile, `package` access ise aynı package ile sınırlar.

English: Using the `protected` keyword means the code is available from the same package or subclasses. Using the `public` keyword means the code is available from anywhere.

Türkçe: `protected`, aynı package'ten veya subclass'lardan erişim sağlar. `public` ise her yerden erişim sağlar.

English: Both static methods and static variables are shared by all instances of the class.

Türkçe: Hem `static` method hem `static` variable, class'ın bütün instance'ları arasında paylaşılır.

English: When referenced from outside the class, they are called using the class name— for example, `Pigeon.fly()`. Instance members are allowed to call static members, but static members are not allowed to call instance members. In addition, static imports are used to import static members.

Türkçe: Class dışından erişilirken `static` member class name ile çağrılır; örneğin `Pigeon.fly()`. Instance member'lar `static` member'ları çağırabilir; ancak `static` member'lar object reference olmadan instance member'ları çağıramaz. `static` member'lar static import ile de import edilebilir.

Source page 0264

English: We also presented the final modifier and showed how it can be applied to local, instance, and static variables. Remember, a local variable is effectively final if it is not modified after it is assigned. One quick test for this is to add the final modifier and see if the code still compiles.

Türkçe: Ayrıca `final` modifier'ın local, instance ve `static` variable'lara nasıl uygulandığını gördük. Local variable, atandıktan sonra değiştirilmiyorsa effectively final'dır. Hızlı test olarak declaration'a `final` ekleyip kodun hâlâ derlenip derlenmediğine bakabilirsiniz.

English: Java uses pass-by-value, which means that calls to methods create a copy of the parameters. Assigning new values to those parameters in the method doesn't affect the caller's variables. Calling methods on objects that are method parameters changes the state of those objects and is reflected in the caller. Java supports autoboxing and unboxing of primitives and wrappers automatically within a method and through method calls.

Türkçe: Java pass-by-value kullanır; method call sırasında argument value'larının kopyaları parameter'lara verilir. Method içinde parameter'a yeni value atamak caller variable'ını etkilemez. Ancak parameter'ın işaret ettiği object üzerinde method çağırmak object state'ini değiştirir ve bu değişiklik caller tarafından görülür. Java ayrıca primitive ve wrapper'lar arasında autoboxing/unboxing yapar.

English: Overloaded methods are methods with the same name but a different parameter list. Java calls the most specific method it can find. Exact matches are preferred, followed by wider primitives. After that comes autoboxing and finally varargs.

Türkçe: Overloaded method'lar aynı name'e, farklı parameter list'e sahiptir. Java bulabildiği en specific method'u seçer: önce exact match, sonra daha geniş primitive, ardından autoboxing ve en son varargs.

English: Make sure you understand everything in this chapter. It sets the foundation of what you learn in the next chapters.

Türkçe: Bu bölümdeki her şeyi anladığınızdan emin olun. Sonraki bölümlerde öğreneceklerinizin temelini oluşturur.

Exam Essentials

Türkçe başlık: Sınav İçin Temel Noktalar

English: Be able to identify correct and incorrect method declarations. Be able to view a method signature and know if it is correct, contains invalid or conflicting elements, or contains elements in the wrong order.

Türkçe: Geçerli ve geçersiz method declaration'ları tanıyabilin. Bir method signature veya declaration'da geçersiz ya da birbiriyle çakışan öğeleri ve yanlış sıralamayı belirleyebilmelisiniz.

English: Identify when a method or field is accessible. Recognize when a method or field is accessible when the access modifier is: private, package (omitted), protected, or public.

Türkçe: Bir method veya field'ın ne zaman erişilebilir olduğunu belirleyin. Access modifier `private`, package (yazılmaz), `protected` veya `public` olduğunda erişim sonucunu tanıyın.

English: Understand how to declare and use final variables. Local, instance, and static variables may be declared final. Be able to understand how to declare them and how they can (or cannot) be used.

Türkçe: `final` variable'ların nasıl bildirildiğini ve kullanıldığını anlayın. Local, instance ve `static` variable'ların `final` bildirilebildiğini; sonrasında nelerin yapılabildiğini ve yapılamadığını bilin.

English: Be able to spot effectively final variables. Effectively final variables are local variables that are not modified after being assigned. Given a local variable, be able to determine if it is effectively final.

Türkçe: Effectively final variable'ları ayırt edin. Bunlar, değer atamasından sonra değiştirilmeyen local variable'lardır. Verilen bir local variable'ın effectively final olup olmadığını belirleyebilmelisiniz.

English: Recognize valid and invalid uses of static imports. Static imports import static members. They are written as `import static`, not `static import`. Make sure they are importing static methods or variables rather than class names.

Türkçe: Static import'un geçerli ve geçersiz kullanımlarını tanıyın. Static import `static` member'ları getirir ve `static import` değil, `import static` biçiminde yazılır. Class name yerine `static` method veya variable import edildiğini doğrulayın.

English: Apply autoboxing and unboxing. The process of automatically converting from a primitive value to a wrapper class is called autoboxing, while the reciprocal process is called unboxing. Watch for a `NullPointerException` when performing unboxing.

Türkçe: Autoboxing ve unboxing'i uygulayın. Primitive value'nun karşılık gelen wrapper class'a otomatik conversion'ı autoboxing, ters yöndeki conversion ise unboxing'dir. `null` reference unbox edilirken `NullPointerException` fırlatılabileceğine dikkat edin.

English: State the output of code involving methods. Identify when to call static rather than instance methods based on whether the class name or object comes before the method. Recognize that instance methods can call static methods and that static methods need an instance of the object in order to call an instance method.

Türkçe: Method içeren code'un output'unu belirleyin. Method'dan önce class name mi yoksa object reference mı kullanıldığına bakarak `static` ve instance method call'larını ayırt edin. Instance method'un `static` method çağırabildiğini; `static` method'un ise instance method çağırması için bir object instance'ına ihtiyaç duyduğunu bilin.

English: Recognize the correct overloaded method. Exact matches are used first, followed by wider primitives, followed by autoboxing, followed by varargs. Assigning new values to method parameters does not change the caller, but calling methods on them does.

Türkçe: Doğru overloaded method'u tanıyın. Seçim sırası exact match, wider primitive, autoboxing ve varargs'tır. Method parameter'ına yeni value atamak caller'ı değiştirmez; parameter'ın gösterdiği object üzerinde method çağırması ise object state'ini değiştirebilir.

Review Questions

Türkçe başlık: İnceleme Soruları

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm inceleme sorularının yanıtlarını Appendix'de bulabilirsiniz.

Question 1 / Soru 1

English: 1. Which statements about the final modifier are correct? (Choose all that apply.)

Türkçe: 1. `final` modifier hakkında hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English: A. Instance and static variables can be marked final.

Türkçe: A. Instance ve `static` variable'lar `final` olarak işaretlenebilir.

English: B. A variable is effectively final only if it is marked final.

Türkçe: B. Bir variable yalnızca `final` olarak işaretlenmişse effectively final olur.

English: C. An object that is marked final cannot be modified.

Türkçe: C. `final` olarak işaretlenmiş bir object değiştirilemez.

English: D. Local variables cannot be declared with type var and the final modifier.

Türkçe: D. Local variable'lar `var` type'ı ve `final` modifier ile bildirilemez.

English: E. A primitive that is marked final cannot be modified.

Türkçe: E. `final` olarak işaretlenen bir primitive değiştirilemez.

Question 2 / Soru 2

English: 2. Which of the following can fill in the blank in this code to make it compile? (Choose all that apply.)

Türkçe: 2. Kodun derlenmesi için boşluğa aşağıdakilerden hangileri gelebilir? (Uygun olanların tümünü seçin.)

```
public class Ant {  
    _____ void method() {}  
}
```

- A. default
- B. final
- C. private
- D. Public
- E. String
- F. zzz:

Question 3 / Soru 3

English: 3. Which of the following methods compile? (Choose all that apply.)

Türkçe: 3. Aşağıdaki method'lardan hangileri derlenir? (Uygun olanların tümünü seçin.)

- A. final static void rain() {}
- B. public final int void snow() {}
- C. private void int hail() {}
- D. static final void sleet() {}
- E. void final ice() {}
- F. void public slush() {}

Question 4 / Soru 4

English: 4. Which of the following can fill in the blank and allow the code to compile? (Choose all that apply.)

Türkçe: 4. Aşağıdakilerden hangileri boşluğu doldurup kodun derlenmesini sağlayabilir? (Uygun olanların tümünü seçin.)

```
final _____ song = 6;
```

- A. int
- B. Integer
- C. long
- D. Long
- E. double
- F. Double

Question 5 / Soru 5

English: 5. Which of the following methods compile? (Choose all that apply.)

Türkçe: 5. Aşağıdaki method'lardan hangileri derlenir? (Uygun olanların tümünü seçin.)

- A. public void january() { return; }
- B. public int february() { return null; }
- C. public void march() {}
- D. public int april() { return 9; }
- E. public int may() { return 9.0; }
- F. public int june() { return; }

Question 6 / Soru 6

English: 6. Which of the following methods compile? (Choose all that apply.)

Türkçe: 6. Aşağıdaki method'lardan hangileri derlenir? (Uygun olanların tümünü seçin.)

- A. public void violin(int... nums) {}
- B. public void viola(String values, int... nums) {}
- C. public void cello(int... nums, String values) {}
- D. public void bass(String... values, int... nums) {}
- E. public void flute(String[] values, ...int nums) {}
- F. public void oboe(String[] values, int[] nums) {}

Question 7 / Soru 7

English: 7. Given the following method, which of the method calls return 2? (Choose all that apply.)

Türkçe: 7. Aşağıdaki method verildiğinde hangi method call'ları 2 döndürür? (Uygun olanların tümünü seçin.)

```
public int juggle(boolean b, boolean... b2) {  
    return b2.length;  
}
```

A. `juggle();`
B. `juggle(true);`
C. `juggle(true, true);`
D. `juggle(true, true, true);`
E. `juggle(true, {true, true});`
F. `juggle(true, new boolean[2]);`

Question 8 / Soru 8

English: 8. Which of the following statements is correct?

Türkçe: 8. Aşağıdaki ifadelerden hangisi doğrudur?

English: A. Package access is more lenient than protected access.

Türkçe: A. Package access, `protected` access'ten daha esnekler.

English: B. A public class that has private fields and package methods is not visible to classes outside the package.

Türkçe: B. `private` field'ları ve package-access method'ları olan `public` bir class, package dışındaki class'lar tarafından görülemez.

English: C. You can use access modifiers so only some of the classes in a package see a particular package class.

Türkçe: C. Access modifier kullanarak bir package'teki class'ların yalnızca bazılarının belirli bir package-access class'ı görmesini sağlayabilirsiniz.

English: D. You can use access modifiers to allow access to all methods and not any instance variables.

Türkçe: D. Access modifier kullanarak bütün method'lara erişim verip hiçbir instance variable'a erişim vermeyebilirsiniz.

English: E. You can use access modifiers to restrict access to all classes that begin with the word Test.

Türkçe: E. Access modifier kullanarak adı `Test` kelimesiyle başlayan bütün class'lara erişimi kısıtlayabilirsiniz.

Question 9 / Soru 9

English: 9. Given the following class definitions, which lines in the main() method generate a compiler error? (Choose all that apply.)

Türkçe: 9. Aşağıdaki class definition'lar verildiğinde `main()` method'undaki hangi satırlar compiler error üretir? (Uygun olanların tümünü seçin.)

```
// Classroom.java
package my.school;
public class Classroom {
    private int roomNumber;
    protected static String teacherName;
    static int globalKey = 54321;
    public static int floor = 3;
    Classroom(int r, String t) {
        roomNumber = r;
        teacherName = t;
    }
}

// School.java
1: package my.city;
2: import my.school.*;
3: public class School {
4:     public static void main(String[] args) {
5:         System.out.println(Classroom.globalKey);
6:         Classroom room = new Classroom(101, "Mrs. Anderson");
7:         System.out.println(room.roomNumber);
8:         System.out.println(Classroom.floor);
9:         System.out.println(Classroom.teacherName); } }
```

English: A. None: the code compiles fine.

Türkçe: A. Hiçbiri: Kod sorunsuz derlenir.

English: B. Line 5

Türkçe: B. 5. satır

English: C. Line 6

Türkçe: C. 6. satır

English: D. Line 7

Türkçe: D. 7. satır

English: E. Line 8

Türkçe: E. 8. satır

English: F. Line 9

Türkçe: F. 9. satır

Question 10 / Soru 10

English: 10. What is the output of executing the Chimp program?

Türkçe: 10. `Chimp` programı çalıştırıldığında output nedir?

```
// Rope.java
1: package rope;
2: public class Rope {
3:     public static int LENGTH = 5;
4:     static {
5:         LENGTH = 10;
6:     }
7:     public static void swing() {
8:         System.out.print("swing ");
9:     } }

// Chimp.java
1: import rope.*;
2: import static rope.Rope.*;
3: public class Chimp {
4:     public static void main(String[] args) {
5:         Rope.swing();
6:         new Rope().swing();
7:         System.out.println(LENGTH);
8:     } }
```

English: A. swing swing 5

Türkçe: A. swing swing 5

English: B. swing swing 10

Türkçe: B. swing swing 10

English: C. Compiler error on line 2 of Chimp

Türkçe: C. `Chimp`'in 2. satırında compiler error

English: D. Compiler error on line 5 of Chimp

Türkçe: D. `Chimp`'in 5. satırında compiler error

English: E. Compiler error on line 6 of Chimp

Türkçe: E. `Chimp`'in 6. satırında compiler error

English: F. Compiler error on line 7 of Chimp

Türkçe: F. `Chimp`'in 7. satırında compiler error

Question 11 / Soru 11

English: 11. Which statements are true of the following code? (Choose all that apply.)

Türkçe: 11. Aşağıdaki kodla ilgili hangi ifadeler doğrudur? (Geçerli olanların tümünü seçin.)

```
1: public class Rope {
2:     public static void swing() {
3:         System.out.print("swing");
4:     }
5:     public void climb() {
6:         System.out.println("climb");
7:     }
8:     public static void play() {
9:         swing();
10:        climb();
11:    }
12:    public static void main(String[] args) {
13:        Rope rope = new Rope();
14:        rope.play();
15:        Rope rope2 = null;
16:        System.out.print("-");
17:        rope2.play();
18:    } }
```

English: A. The code compiles as is.

Türkçe: A. Kod olduğu gibi derlenir.

English: B. There is exactly one compiler error in the code.

Türkçe: B. Kodda tam olarak bir derleme hatası vardır.

English: C. There are exactly two compiler errors in the code.

Türkçe: C. Kodda tam olarak iki derleme hatası vardır.

English: D. If the line(s) with compiler errors are removed, the output is `swing-climb`.

Türkçe: D. Derleme hatası bulunan satırlar kaldırılırsa çıktı `swing-climb` olur.

English: E. If the line(s) with compiler errors are removed, the output is `swing-swing`.

Türkçe: E. Derleme hatası bulunan satırlar kaldırılırsa çıktı `swing-swing` olur.

English: F. If the line(s) with compile errors are removed, the code throws a `NullPointerException`.

Türkçe: F. Derleme hatası bulunan satırlar kaldırılırsa kod `NullPointerException` fırlatır.

Question 12 / Soru 12

English: 12. How many variables in the following method are effectively final?

Türkçe: 12. Aşağıdaki method'da kaç variable effectively final'dır?

```
10: public void feed() {  
11:     int monkey = 0;  
12:     if(monkey > 0) {  
13:         var giraffe = monkey++;  
14:         String name;  
15:         name = "geoffrey";  
16:     }  
17:     String name = "milly";  
18:     var food = 10;  
19:     while(monkey <= 10) {  
20:         food = 0;  
21:     }  
22:     name = null;  
23: }
```

English: A. 1

Türkçe: A. 1

English: B. 2

Türkçe: B. 2

English: C. 3

Türkçe: C. 3

English: D. 4

Türkçe: D. 4

English: E. 5

Türkçe: E. 5

English: F. None of the above. The code does not compile.

Türkçe: F. Yukarıdakilerin hiçbiri. Kod derlenmez.

Question 13 / Soru 13

English: 13. What is the output of the following code?

Türkçe: 13. Aşağıdaki kodun çıktısı nedir?

```
// RopeSwing.java
import rope.*;
import static rope.Rope.*;
public class RopeSwing {
    private static Rope rope1 = new Rope();
    private static Rope rope2 = new Rope();
    {
        System.out.println(rope1.length);
    }
    public static void main(String[] args) {
        rope1.length = 2;
        rope2.length = 8;
        System.out.println(rope1.length);
    }
}

// Rope.java
package rope;
public class Rope {
    public static int length = 0;
}
```

English: A. 02

Türkçe: A. 02

English: B. 08

Türkçe: B. 08

English: C. 2

Türkçe: C. 2

English: D. 8

Türkçe: D. 8

English: E. The code does not compile.

Türkçe: E. Kod derlenmez.

English: F. An exception is thrown.

Türkçe: F. Bir exception fırlatılır.

Question 14 / Soru 14

English: 14. How many lines in the following code have compiler errors?

Türkçe: 14. Aşağıdaki kodda kaç satırda derleyici hatası var?

```
1: public class RopeSwing {
2:     private static final String leftRope;
3:     private static final String rightRope;
4:     private static final String bench;
5:     private static final String name = "name";
6:     static {
7:         leftRope = "left";
8:         rightRope = "right";
9:     }
10:    static {
11:        name = "name";
12:        rightRope = "right";
13:    }
14:    public static void main(String[] args) {
15:        bench = "bench";
16:    }
17: }
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 2

Türkçe: C. 2

English: D. 3

Türkçe: D. 3

English: E. 4

Türkçe: E. 4

English: F. 5

Türkçe: F. 5

Question 15 / Soru 15

English: 15. Which of the following can replace line 2 to make this code compile? (Choose all that apply.)

Türkçe: 15. Bu kodun derlenmesi için 2. satırın yerine aşağıdakilerden hangileri gelebilir? (Geçerli olanların tümünü seçin.)

```
1: import java.util.*;
2: // INSERT CODE HERE
3: public class Imports {
4:     public void method(ArrayList<String> list) {
5:         sort(list);
6:     }
7: }
```

```
A. import static java.util.Collections;  
B. import static java.util.Collections.*;  
C. import static java.util.Collections.sort (ArrayList<String>);  
D. static import java.util.Collections;  
E. static import java.util.Collections.*;  
F. static import java.util.Collections.sort (ArrayList<String>);
```

Question 16 / Soru 16

English: 16. What is the result of the following statements?

Türkçe: 16. Aşağıdaki ifadelerin sonucu nedir?

```
1: public class Test {  
2:     public void print(byte x) {  
3:         System.out.print("byte-");  
4:     }  
5:     public void print(int x) {  
6:         System.out.print("int-");  
7:     }  
8:     public void print(float x) {  
9:         System.out.print("float-");  
10:    }  
11:    public void print(Object x) {  
12:        System.out.print("Object-");  
13:    }  
14:    public static void main(String[] args) {  
15:        Test t = new Test();  
16:        short s = 123;  
17:        t.print(s);  
18:        t.print(true);  
19:        t.print(6.789);  
20:    }  
21: }
```

English: A. byte-float-Object-

Türkçe: A. byte-float-Object-

English: B. int-float-Object-

Türkçe: B. int-float-Object-

English: C. byte-Object-float-

Türkçe: C. byte-Object-float-

English: D. int-Object-float-

Türkçe: D. int-Object-float-

English: E. int-Object-Object-

Türkçe: E. int-Object-Object-

English: F. byte-Object-Object-

Türkçe: F. byte-Object-Object-

Question 17 / Soru 17

English: 17. What is the result of the following program?

Türkçe: 17. Aşağıdaki programın sonucu nedir?

```
1: public class Squares {  
2:     public static long square(int x) {  
3:         var y = x * (long) x;  
4:         x = -1;  
5:         return y;  
6:     }  
7:     public static void main(String[] args) {  
8:         var value = 9;  
9:         var result = square(value);  
10:        System.out.println(value);  
11:    } }
```

English: A. -1

Türkçe: A. -1

English: B. 9

Türkçe: B. 9

English: C. 81

Türkçe: C. 81

English: D. Compiler error on line 9

Türkçe: D. 9. satırda derleme hatası

English: E. Compiler error on a different line

Türkçe: E. Farklı bir satırda derleme hatası

Question 18 / Soru 18

English: 18. Which of the following are output by the following code? (Choose all that apply.)

Türkçe: 18. Aşağıdaki code hangi seçenekleri output olarak üretir? (Uygun olanların tümünü seçin.)

```
public class StringBuilders {
    public static StringBuilder work(StringBuilder a,
        StringBuilder b) {
        a = new StringBuilder("a");
        b.append("b");
        return a;
    }
    public static void main(String[] args) {
        var s1 = new StringBuilder("s1");
        var s2 = new StringBuilder("s2");
        var s3 = work(s1, s2);
        System.out.println("s1 = " + s1);
        System.out.println("s2 = " + s2);
        System.out.println("s3 = " + s3);
    }
}
```

English: A. s1 = a

Türkçe: A. s1 = a

English: B. s1 = s1

Türkçe: B. s1 = s1

English: C. s2 = s2

Türkçe: C. s2 = s2

English: D. s2 = s2b

Türkçe: D. s2 = s2b

English: E. s3 = a

Türkçe: E. s3 = a

English: F. The code does not compile.

Türkçe: F. Kod derlenmez.

Question 19 / Soru 19

English: 19. Which of the following will compile when independently inserted in the following code?
(Choose all that apply.)

Türkçe: 19. Aşağıdaki code'a birbirinden bağımsız olarak eklendiğinde seçeneklerden hangileri derlenir?
(Uygun olanların tümünü seçin.)

```
1: public class Order3 {
2:     final String value1 = "red";
3:     static String value2 = "blue";
4:     String value3 = "yellow";
5:     {
6:         // CODE SNIPPET 1
7:     }
8:     static {
9:         // CODE SNIPPET 2
10:    } }
```

A. Insert at line 6: value1 = "green";
B. Insert at line 6: value2 = "purple";
C. Insert at line 6: value3 = "orange";
D. Insert at line 9: value1 = "magenta";
E. Insert at line 9: value2 = "cyan";
F. Insert at line 9: value3 = "turquoise";

Türkçe seçenekler: A. 6. satıra value1 = "green"; ekleyin. B. 6. satıra value2 = "purple"; ekleyin. C. 6. satıra value3 = "orange"; ekleyin. D. 9. satıra value1 = "magenta"; ekleyin. E. 9. satıra value2 = "cyan"; ekleyin. F. 9. satıra value3 = "turquoise"; ekleyin.

Question 20 / Soru 20

English: 20. Which of the following are true about the following code? (Choose all that apply.)

Türkçe: 20. Aşağıdaki kodla ilgili hangi ifadeler doğrudur? (Geçerli olanların tümünü seçin.)

```
public class Run {
    static void execute() {
        System.out.print("1-");
    }
    static void execute(int num) {
        System.out.print("2-");
    }
    static void execute(Integer num) {
        System.out.print("3-");
    }
    static void execute(Object num) {
        System.out.print("4-");
    }
    static void execute(int... nums) {
        System.out.print("5-");
    }
    public static void main(String[] args) {
        Run.execute(100);
        Run.execute(100L);
    }
}
```

English: A. The code prints out 2-4-.

Türkçe: A. Kod 2-4- çıktısını üretir.

English: B. The code prints out 3-4-.

Türkçe: B. Kod 3-4- çıktısını üretir.

English: C. The code prints out 4-2-.

Türkçe: C. Kod 4-2- çıktısını üretir.

English: D. The code prints out 4-4-.

Türkçe: D. Kod 4-4- çıktısını üretir.

English: E. The code prints 3-4- if you remove the method `static void execute(int num)`.

Türkçe: E. `static void execute(int num)` method'u kaldırılırsa kod 3-4- çıktısını üretir.

English: F. The code prints 4-4- if you remove the method `static void execute(int num)`.

Türkçe: F. `static void execute(int num)` method'u kaldırılırsa kod 4-4- çıktısını üretir.

Question 21 / Soru 21

English: 21. Which method signatures are valid overloads of the following method signature? (Choose all that apply.)

Türkçe: 21. Hangi method signature'lar aşağıdaki method signature'ın geçerli overload'larıdır? (Uygun olanların tümünü seçin.)

```
public void moo(int m, int... n)
```

- A. `public void moo(int a, int... b)`
- B. `public int moo(char ch)`
- C. `public void moooo(int... z)`
- D. `private void moo(int... x)`
- E. `public void moooo(int y)`
- F. `public void moo(int... c, int d)`
- G. `public void moo(int... i, int j...)`

Kapsam doğrulaması

Kapsam özeti: 0219–0274 aralığındaki 56/56 kaynak sayfa doğrulandı; eksik sayfa yoktur.

Appendix · Kaynak dışı teknik pekiştirme

Kaynak bölümün çevirisine ait olmayan önceki OCP pekiştirme içeriği ayrı ana kaynakta korunmuştur: [Unit 05 technical memory notes](#).